

Portfolio Project - Stage 3 Report

Technical Documentation

Table of content:

1. Summary.....	2
2. User Stories.....	3
2.1. Must have.....	3
2.2. Should have.....	4
2.3. Could have.....	5
2.4. Won't have.....	6
2.5. Notes.....	8
3. Design System Architecture.....	9
3.1. Backend.....	9
3.2. Frontend.....	11
3.3. Database.....	13
4. External APIs.....	15
4.1. Film Metadata API.....	15
4.2. Streaming Availability API.....	17
4.3. LLM API.....	17
5. System Architecture Diagrams.....	20
5.1. High-Level Architecture Overview.....	21
5.2. Detailed Component Architecture.....	22
5.3. Design Patterns.....	24
6. Backend Components, Classes, and Database Design.....	25
6.1. Class Diagram.....	25
6.2. ER Diagram.....	33
7. Endpoints and URLs.....	36
7.1. External Endpoints.....	36
7.2. Internal Endpoints.....	38
8. Frontend Components and Design.....	40
8.1. React Frontend Architecture.....	41
8.2. React Components.....	42
8.3. Frontend Interactions.....	44
8.4. Mockup.....	47
9. Sequence Diagrams.....	53
9.1. User Registration.....	54

9.2. User Login.....	56
9.3. Searching for a Movie.....	58
9.4. Log a Film.....	60
9.5. Film Recommendation.....	62
10. SCM and QA Strategy.....	64
10.1. Source Control Management (SCM).....	64
10.2. Quality Assurance (QA) Strategy.....	65
10.3. Development Pipeline.....	67
11. Tech Stack Summary and Justifications.....	68
11.1. Technical Stack, Tools and APIs.....	68
11.2. Technical justifications:.....	69

1. Summary

This document constitutes the Stage 3 Technical Documentation for CinéMood, a personal film diary and AI-powered recommendation web application developed as a solo end-of-year portfolio project at Holberton School Bordeaux.

The purpose of this document is to translate the MVP concept defined in Stage 1 into a concrete and structured technical blueprint, covering all aspects of the system before any code is written. It includes the following sections:

- User Stories: a prioritised list of functional requirements from the user's perspective, classified using the MoSCoW method.
 - System Architecture: a high-level diagram showing how the frontend, backend, database, and external services interact.
 - Components, Classes, and Database Design: the internal structure of the system, including the data model and key backend components.
 - Sequence Diagrams: step-by-step interaction flows for the key use cases of the application.
 - API Specifications: documentation of the external APIs used and the internal API endpoints defined for the project.
 - SCM and QA Strategies: the source control and testing approach planned for the development phase.
 - Technical Justifications: the rationale behind each major technical choice.
-

2. User Stories

2.1. Must have

This section lists the core features of CinéMood, classified as “Must Have” using the **MoSCoW** method.

These user stories represent the minimum viable product: without them, the application cannot function or be presented.

They cover the essential user journey - from account creation to film logging - and form the foundation on which all other features are built.

Development will prioritise these stories above all others.

User Story	Title	Description
US-01	Create an account	As a new user , I want to create an account by providing my personal information (name, email, password, and streaming platform preferences), so that my data is saved and personalised from the start.
US-02	Log in	As a returning user , I want to log into my account , so that I can access my viewing history , my watchlist , and my personalised recommendations .
US-03	Search for a film	As a user , I want to search for a film by title , so that I can quickly find it and either log it or add it to my watchlist .
US-04	Log a film and rate it with tags	As a user , I want to log a film I have just watched and rate it using emotional and contextual tags (chosen from a predefined list or popular tags for that film), so that my rating reflects how I truly experienced it, rather than just a score out of 5.
US-05	View my dashboard	As a user , I want to see my viewing history and my watchlist on a

User Story	Title	Description
		personal dashboard , so that I can browse what I have watched , how I rated it, and what I plan to watch next.
US-06	Add a film to my watchlist	As a user , I want to indicate that I want to watch a film (without having seen it yet), so that it is saved to my “watchlist” and I can find it easily later.

2.2. Should have

This section details features that significantly enhance the user experience and are strongly expected in the final MVP, but whose absence would not prevent the core product from functioning.

User Story	Title	Description
US-07	Access the recommendation experience	As a user , I want to tap a prominent button on the home screen to start the recommendation experience , so that I am guided through a mood questionnaire and then film discovery in a single, fluid flow .
US-08	Discover films through swiping	As a user , I want to swipe through film cards (right = interested, left = not interested) after completing the mood questionnaire , so that I can give preference signals in a fun and intuitive way. Each card shows the film poster , a few stills , a very short synopsis , and popular tags or highlight badges (e.g. "Won 3 Oscars", "Cult classic", "Mind-blowing ending").
US-09	Receive personalised recommendations	As a user , I want to receive a curated list of film recommendations at the end of the swipe session ,

User Story	Title	Description
		so that I can choose what to watch based on my mood and taste. I can select a film to add it directly to my watchlist .
US-10	Set my streaming platform preferences	As a user , I want to indicate which streaming platforms I have access to in my profile , so that all recommendations are filtered to films I can actually watch easily.
US-11	See streaming availability	As a user , I want to see which streaming platform a recommended film is available on (displayed with the platform logo), so that I can instantly know where to watch it without having to browse through every catalog myself.

2.3. Could have

This section lists features that would enrich the product and improve user engagement, but are not critical for the MVP.

They will be implemented only if time allows after the **Must Have** and **Should Have** features are complete.

User Story	Title	Description
US-12	Classify a film with a prestige tier	As a user , I want to assign a prestige tier to a film I have watched (such as Platinum, Gold, Silver, Bronze, or Trash...), so that I have a global rating that complements my tags and gives me a quick visual overview of my favourites .
US-13	Write a personal review	As a user , I want to add a personal note or written review when logging a film, so that I can remember my thoughts in my own words and build a real personal film journal .

User Story	Title	Description
US-14	Receive age-appropriate recommendations	As a user, I would like to enter my age in my personal user information so that I can receive movie recommendations appropriate for my age group .
US-15	See film details before logging	As a user , I want to see a film's details (poster, synopsis, director, cast, runtime, and external ratings such as Rotten Tomatoes or AlloCiné displayed as contextual information) before logging it , so that I can confirm I have the right film and get a neutral reference point alongside my own tags.
US-16	Be notified to rate a film from my watchlist	As a user , I want to receive a notification the day after adding a film to my watchlist (suggesting I may have watched it), so that I am prompted to log and rate it while my impressions are still fresh.

2.4. Won't have

This section documents features that were considered during the brainstorming phase but are explicitly excluded from the MVP scope.

They are not abandoned - they represent a documented roadmap for future versions of CinéMood.

User Story	Title	Description
US-17	Follow other users	As a user, I want to follow other users and see their ratings, recommendations, tags and reviews, so that I can discover films through people I trust. Excluded from MVP: requires a social graph and significant backend complexity. Planned for a future version.
US-18	Get recommendations for films currently showing nearby	As a user, I want to receive film recommendations that include current cinema releases (possibly near me), so that the app helps me decide what to go and see as well as what to stream. Excluded from MVP: depends on geolocation and cinema listing APIs with limited availability. Planned for a future version.
US-19	Share a film recommendation	As a user, I want to share a film recommendation or my watchlist with a friend, so that we can decide what to watch together. Excluded from MVP: requires sharing/social features. Planned for a future version.
US-20	Export my viewing history	As a user, I want to export my viewing history as a file, so that I can keep a personal archive of everything I have watched. Excluded from MVP: low priority for initial release. Planned for a future version.
US-21	TV schedule recommendations	As a user, I want my recommendations to also take into account the TV schedule on national channels, so that I can get recommendations based on what's currently airing on TV. Excluded from MVP: low priority for initial release. The feasibility of this feature is not guaranteed.

User Story	Title	Description
US-22	Duplicate features for series	As a user, I want to access all the app's features, but also for TV shows and series, so I can get personalized recommendations and keep track of the episodes I haven't watched yet. Excluded from MVP: beyond the scope of the app, the core of the project focuses on movies/feature-length films.
US-23	Guest mode navigation	As a visitor, I want to browse films and get recommendations without creating an account, so that I can discover the app before committing to registration. A sign-up prompt appears at the end of the recommendation flow to save results. Excluded from MVP: requires managing dual authenticated/unauthenticated states across the entire frontend. Planned as a priority for the next version.

2.5. Notes

- The app is designed as a responsive web application from the start (desktop and mobile). This is a technical requirement, not a separate user story.
- External ratings (Rotten Tomatoes, AlloCiné) are displayed as neutral contextual information only. The core rating system of CinéMood is based on tags and prestige tiers, not numerical scores.
- The watchlist and the viewing history are two distinct sections of the dashboard, clearly separated in the UI.

3. Design System Architecture

CinéMood follows a classic **three-tier architecture**, separating the **user interface**, the **business logic**, and the **data layer** into distinct components.

3.1. Backend

Python + FastAPI

The backend is the core of CinéMood's business logic.

It exposes a REST API consumed by the frontend, handles authentication, manages all data exchanges with the database, and orchestrates calls to external services.

Why Python?

Python was chosen over Node.js as the backend language for three main reasons.

- First, it is the language covered throughout the Holberton curriculum, which reduces the learning curve and allows more focus on architecture and product quality.
 - Second, Python is the dominant language in the AI and machine learning ecosystem - libraries like scikit-learn, pandas, or LangChain are all Python-native, which keeps the door open for deeper ML integration in future versions.
 - Third, Python combined with FastAPI is increasingly in demand in the Bordeaux tech market, particularly in companies building data-driven or AI-assisted products.
-

Why FastAPI?

Flask was considered a serious alternative - it is simpler, extremely well documented, and widely used in the industry.

However, FastAPI offers three concrete advantages that justify the choice for this project:

- **Automatic data validation:** FastAPI uses Pydantic models to validate incoming request data automatically. If a request is malformed, FastAPI returns a clear error without any manual handling. With Flask, this must be implemented manually.
- **Automatic interactive documentation:** FastAPI generates a Swagger UI at “/docs” out of the box. During development, every endpoint can be tested directly in the browser without any external tool. This significantly accelerates the development and debugging workflow.
- **Native async support:** FastAPI is built around Python's “*async/await*” syntax. For CinéMood, this is particularly useful in the recommendation flow, where the backend needs to call several APIs simultaneously. With async, these calls can be made in parallel rather than sequentially, which improves response time noticeably.

Criterion	Flask	FastAPI
Learning curve	Very gentle	Gentle
Data validation	Manual	Automatic (Pydantic)
API documentation	Manual	Automatic (Swagger UI)
Performance	Good	Very good
Async support	Possible but complex	Native
Industry adoption	Very high	Growing rapidly

Backend structure

The backend is organised around four main services:

- **Auth Service:** handles user registration, login, and JWT token generation and validation
 - **Film Service:** manages film search (via TMDB API), logging, tagging, watchlist management, and viewing history
 - **User Service:** manages user profiles, streaming platform preferences, and prestige tier assignments
 - **Recommendation Service:** orchestrates the mood questionnaire logic, swipe session data, and calls to the LLM API and movie database API to generate personalized recommendations
-

3.2. Frontend

React + Tailwind CSS

The frontend is a **React-based web application** using **React Router** to handle navigation between pages.

Each view (dashboard, film search, recommendation experience, user profile) is rendered as a React component and displayed according to the current URL, without triggering full page reloads.

This produces a fast and fluid user experience, which is particularly important for CinéMood's interactive features such as the swipe interface and the mood questionnaire.

Why React?

React is a JavaScript-based UI framework developed by Meta. It works around the concept of **reusable components** - the interface is built as a collection of independent building blocks (a film card, a tag chip, a dashboard section) that can be composed and reused throughout the application.

This makes the codebase easier to maintain and extend over time.

React was chosen for three reasons:

- First, its component-based architecture is well suited to CinéMood's UI, which relies heavily on reusable cards, tags, and interactive elements.
- Second, it is the most widely adopted frontend framework in the industry, and the most in demand on the Bordeaux job market - making it a direct asset for a work-study placement.

Other frameworks were considered but set aside.

- **Vue.js** is simpler to learn but less present on the local job market.
- **Angular** is powerful but significantly over-engineered for a project of this scope.
- **Vanilla JavaScript** would work technically but becomes hard to maintain as soon as the interface grows in complexity.

Why Tailwind CSS?

Tailwind CSS is a utility-first CSS framework.

Rather than writing CSS in separate stylesheets, utility classes are applied directly in the component markup.

For example “**className=“bg-gray-900 text-white rounded-lg p-4”**” instead of defining a separate “**.card**” class in a CSS file.

A common concern with this approach is class repetition. In practice, this is largely solved by React's component model: a film card's styles are written once inside the **FilmCard** component and reused everywhere without duplication. For cases where a named class is preferable, Tailwind's “**@apply**” directive allows grouping utility classes under a custom class name. Tailwind was chosen over **Bootstrap** because it allows a fully custom visual identity - Bootstrap tends to produce recognisable generic interfaces, whereas Tailwind gives complete design freedom, which is important for CinéMood's cinematic and immersive aesthetic.

3.3. Database

PostgreSQL

All persistent data is stored in a PostgreSQL relational database. This includes users, viewing history, watchlists, tags, prestige tiers, and streaming platform preferences.

Why a relational database?

CinéMood's data is deeply interconnected - a user has a viewing history, a history contains films, films have tags, tags belong to users, users have streaming platform preferences. This level of relationships between entities is exactly what relational databases are designed for. A NoSQL solution such as **MongoDB** was considered but quickly set aside: forcing highly relational data into a document model creates unnecessary complexity and makes queries harder to write and maintain.

Why PostgreSQL?

MySQL was a natural candidate given its presence in the Holberton curriculum. However, PostgreSQL was chosen for three reasons.

- First, PostgreSQL is entirely open source with no commercial entity behind it, unlike MySQL which has been owned by Oracle since 2010. This ensures long-term stability and a fully community-driven ecosystem.
 - Second, PostgreSQL is more powerful on complex queries - it handles multiple joins, nested queries, and advanced data types more robustly than MySQL. For CinéMood's data model, which involves several interconnected tables, this is a concrete advantage.
 - Third, PostgreSQL is the standard pairing for FastAPI in modern stacks, and is increasingly demanded by tech companies in the Bordeaux market. Choosing it is both a technically sound decision and a direct asset for employability.
-

Cascade operations

PostgreSQL's support for cascade operations is particularly useful for CinéMood.

When a user deletes their account, all associated data - viewing history, watchlist entries, tags, ratings - can be automatically removed through cascading foreign key constraints, without requiring manual deletion logic in the application code.

This keeps the backend cleaner and reduces the risk of orphaned data in the database.

4. External APIs

CinéMood integrates three external services to power its core features. Each was selected after evaluating available alternatives based on cost, coverage, language support, and ease of integration.

4.1. Film Metadata API

TMDB (The Movie Database)

Film APIs are used to provide all film metadata throughout the application: titles, posters, backdrops, synopsis, genres, director, cast, runtime, and release year.

Data quality, completeness, and query costs were the criteria used to determine which API to select.

To ensure that the app could be rolled out in multiple languages (at least in English and French), the ability to aggregate data in both languages using a single API was also a key factor.

Several options were evaluated:

Criterion	TMDB	OMDb	IMDb API
Free tier	Yes	Limited (1000/day)	~\$150,000/year
French language support	Native	No	No
Posters / images	Yes	Paid only	Yes
Genres	Yes	Yes	Yes
Cast & director	Yes	Yes	Yes
Request limit	40 / 10 seconds	1,000 / day	N/A
Data quality	Excellent	Good	Excellent

TMDB was selected for three reasons:

- First, it is entirely free for non-commercial use with a very generous rate limit.
- Second, titles, synopses, and genres can natively be returned in French via the “**language=fr-FR**” parameter, a feature planned for a post-MVP release. For the MVP, all data is returned in English (**language=en-US**).
- Third, it provides high-quality visual assets (posters, stills) that are central to the film card interface.

OMDb was set aside due to its daily request limit, lack of French support, and paid-only poster access.

The official **IMDb** API was immediately ruled out due to its prohibitive commercial pricing.

Attribution note: as required by TMDB's terms of use, the application will display the mention "This product uses the TMDB API but is not endorsed or certified by TMDB" in its about section.

4.2. Streaming Availability API

TMDB Watch Providers (The Movie Database)

The core of the app requires an API that provides movie availability based on local streaming catalogs in order to generate personalized recommendations.

Three options were evaluated:

Criterion	TMDB Watch Providers	Watchmode	JustWatch
Free tier	Included in TMDB	2,500 calls/month	Commercial only
France coverage	Yes	To be confirmed	Excellent
Integration complexity	Already integrated	Separate API	Not accessible
Data exhaustiveness	Correct	Very good	Excellent

To display which streaming platforms a recommended film is available on, TMDB offers a built-in “/watch/providers” endpoint that returns availability data by country, including France.

TMDB Watch Providers was selected for the MVP for two main reasons:

- It is already included in the TMDB integration, meaning no additional API key, no additional cost, and significantly reduced complexity.
- It also covers France natively, which directly addresses the project's localisation requirement.

Watchmode remains documented as a potential upgrade if TMDB's streaming data proves insufficient in practice - it covers over 200 platforms and offers 2,500 free calls per month.

JustWatch, despite being the most comprehensive option, is a commercial API not accessible to independent developers.

4.3. LLM API

The recommendation engine uses a large language model to process the user's mood input and viewing history and generate personalized film suggestions.

No model training is involved - the integration relies entirely on prompt engineering: a carefully crafted system prompt defines the model's behaviour and output format, and the user's context (mood, history, platform preferences) is passed with each request.

Example system prompt structure:

```
System:
You are a film recommendation assistant.
You must only suggest films – never anything else.

User:
Current mood: [user mood input]
Viewing history: [list of films already watched]
Available platforms: [user streaming preferences]
Suggest 3 films.
```

Note: This example illustrates the conceptual structure of the prompt sent to the LLM: a system instruction that strictly limits the model to film recommendations, combined with a user context built dynamically from the session data. The exact phrasing, output format, and contextual signals (such as swipe results or age preferences) will be refined iteratively during development. A more complete implementation example is provided in [section 7.1](#).

Three providers were evaluated:

Criterion	Mistral AI	OpenAI	Google Gemini
Free tier	Yes (Experiment)	Limited	Most generous
Price (small model)	\$0.10/M tokens	\$0.15/M tokens	\$0.10/M tokens
Data hosting	France / EU	USA	USA
GDPR compliance	Native	Complex	Complex
Documentation quality	Good	Excellent	Good
Output quality	Very good	Excellent	Very good

Mistral AI was selected as the primary provider for three reasons:

- It is a French company with servers hosted in France, making it natively GDPR-compliant - a structural advantage for a European application without any additional configuration.
- Its pricing is among the most competitive on the market (Mistral Small at \$0.10 per million input tokens).
- Finally, supporting a European AI provider is a deliberate choice aligned with the project's values.

Google Gemini is documented as a backup option - its free tier is the most generous available and can be used during development to preserve Mistral credits.

Note on costs: API prices in 2026 have dropped significantly compared to previous years. For a student project with a limited number of test users, free tier quotas are more than sufficient throughout the development phase. Response caching on the backend will further reduce the number of API calls.

5. System Architecture Diagrams

CinéMood follows a classic three-tier architecture, separating the **user interface**, the **business logic**, and the **data layer** into distinct components.

The three tiers are structured as follows:

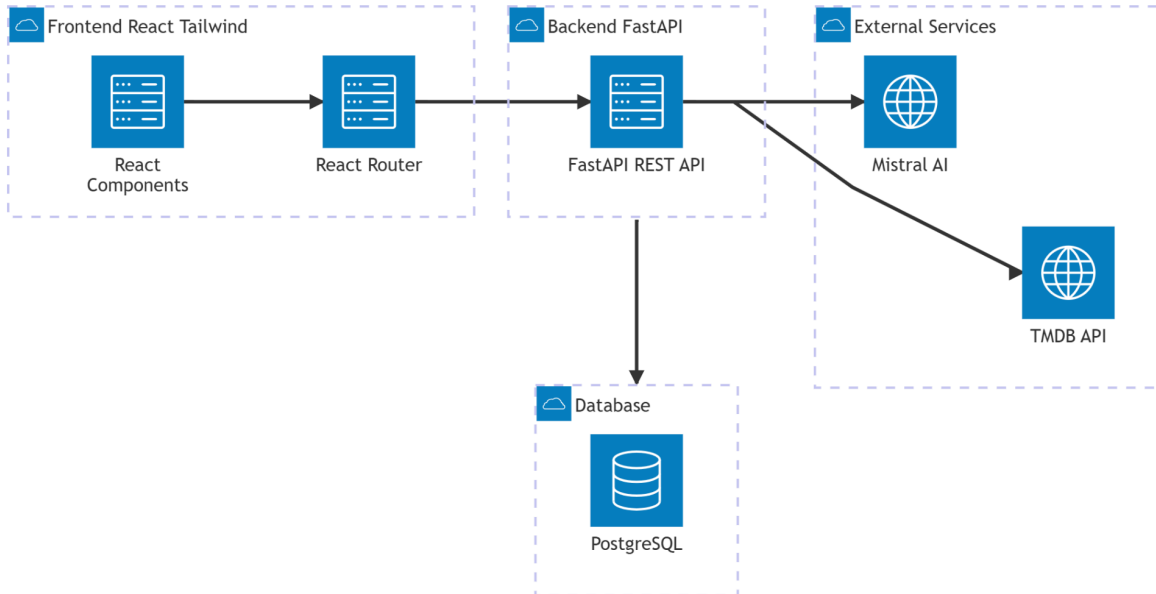
- Tier 1 - **Presentation layer**:
 - The React frontend, running in the user's browser. It handles all user interactions and communicates with the backend exclusively through HTTP requests.
- Tier 2 - **Business logic layer**:
 - The Python/FastAPI backend. It processes requests from the frontend, applies the application logic, and orchestrates calls to the database and external services.
- Tier 3 - **Data layer**:
 - The PostgreSQL database. It stores all persistent data - users, viewing history, watchlists, and tags.

External services (TMDB and Mistral AI) sit outside this three-tier model and are called exclusively by the backend, never directly by the frontend. This keeps API keys secure and centralises all external communication in one place.

Two architecture diagrams are provided below: a **high-level overview** showing the four main components and their connections, and a **detailed view** zooming into the internal structure of the backend - its routes, services, repositories, and design patterns.

5.1. High-Level Architecture Overview

This diagram shows the four main components of the system and how they communicate.

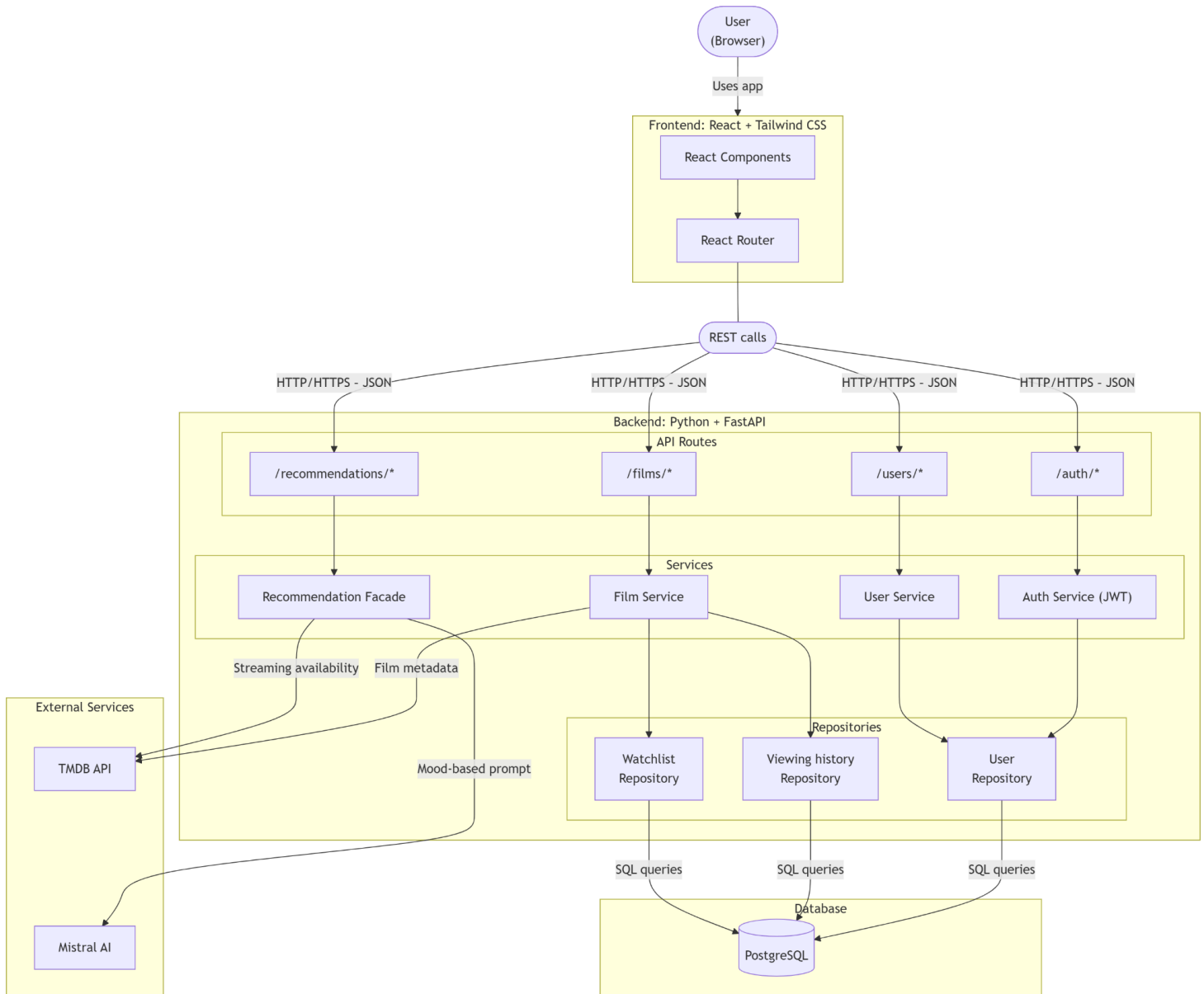


Component Interactions

From	To	Description
React Components	React Router	User actions trigger client-side navigation
React Router	FastAPI REST API	HTTP/HTTPS requests with JSON payloads
FastAPI REST API	PostgreSQL	SQL queries for all persistent data (users, viewing history, watchlist, tags)
FastAPI REST API	TMDB API	Fetches film metadata (title, poster, synopsis, cast) and streaming availability
FastAPI REST API	Mistral AI	Sends mood and viewing history as a prompt, receives film recommendations in JSON

5.2. Detailed Component Architecture

This diagram zooms into the internal structure of the backend, showing the API routes, services, repositories, and their connections to the database and external services.



Component Interactions

From	To	Description
User	React Components	The user interacts with the app through the browser
React Components	React Router	Navigation between pages is handled client-side without full page reloads
React Router	API Routes	All backend communication goes through HTTP/HTTPS calls with JSON payloads
/auth/*	Auth Service	Handles registration, login, and JWT token generation and validation
/films/*	Film Service	Handles film search, logging, tagging, and watchlist management
/users/*	User Service	Handles user profile and streaming platform preferences
/recommendations/*	Recommendation Facade	Orchestrates the full recommendation flow (mood + swipe + LLM)
Auth Service	User Repository	Reads and writes user authentication data
Film Service	Viewing History Repository	Reads and writes film logging and tag data
Film Service	Watchlist Repository	Reads and writes watchlist entries
Film Service	TMDB API	Fetches film metadata (poster, synopsis, cast, genres, runtime) and streaming availability
Recommendation Facade	TMDB API	Fetches streaming availability for recommended films
Recommendation Facade	Mistral AI	Sends a structured prompt with mood and viewing history, receives film suggestions in JSON
All Repositories	PostgreSQL	All persistent data is stored and retrieved via SQL queries

5.3. Design Patterns

Pattern	Where applied	Reason
Repository	Data access layer	Isolates all database queries from business logic. Each entity (User, Viewing history, Watchlist) has its own repository, making the code easier to maintain and test.
Facade	Recommendation Service	Hides the complexity of orchestrating user data and external APIs (TMDB for the films and Mistral AI for the LLM) behind a single clean interface. The route only calls one method - the facade handles everything else.
REST	API design	Standard architectural style for web APIs - stateless*, resource-based, JSON responses. Consumed by the React frontend.

**Stateless means the server never stores session data - authentication is handled via JWT tokens passed in each request.*

6. Backend Components, Classes, and Database Design

This section details the internal structure of the CinéMood system. It covers two complementary views of the application's data model:

- The **class diagram**, which describes the business logic layer - the Python classes, their attributes, methods, and relationships.
- The **ER diagram**, which describes the database layer - the PostgreSQL tables, their columns, and the foreign key relationships between them.

Together, these three views form a complete blueprint of how data is structured, stored, and manipulated in CinéMood.

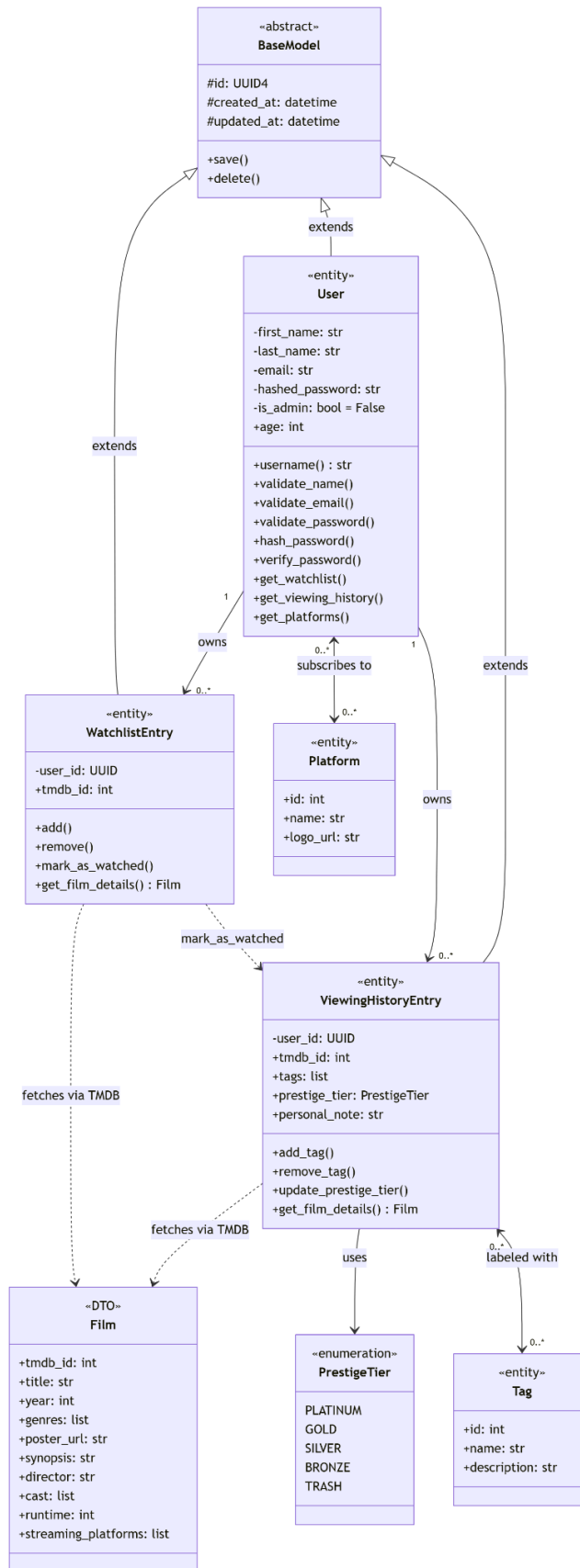
6.1. Class Diagram

The class diagram represents the backend business logic layer of CinéMood. It shows the Python classes used to model the application's data and behaviour, before any interaction with the database or external APIs.

In the class descriptions below, each attribute and method is prefixed with a visibility modifier following UML conventions. These modifiers reflect design intent and document how each element is meant to be accessed within the application:

- **+** Public - accessible from anywhere in the application. Used for methods that services or the frontend need to call, and for non-sensitive attributes.
- **-** Private - accessible only within the class itself. Used for sensitive data such as passwords, foreign keys, and internal identifiers that should never be exposed directly.
- **#** Protected - accessible within the class and its subclasses. Used exclusively for the attributes inherited from BaseModel (id, created_at, updated_at), which every entity can read but which are managed centrally.

Note: Python does not enforce these access levels at runtime. They are used here as a documentation convention to communicate architectural intent, as recommended by the supervising software engineer.



BaseModel <<abstract>>

The base class, inherited by all persistent entities. It centralises three shared attributes and two CRUD methods, avoiding duplication across the codebase.

	Visibility	Name	Type / Return	Description
A T T R I B U T E S	#	id	UUID4	Primary key, generated automatically at creation. Never modified.
	#	created_at	datetime	Timestamp of creation, set once and never modified.
	#	updated_at	datetime	Timestamp of last modification, refreshed automatically by save() .
M E T H O D S	+	save()	None	Creates or updates the entity in the database. Automatically refreshes updated_at on every call.
	+	delete()	None	Permanently removes the entity. PostgreSQL cascade constraints automatically remove all related records.

User <<entity>>

Represents a registered user of CinéMood. Handles authentication, profile data, and access to personal content (viewing history, watchlist, platform preferences).

	Visibility	Name	Type / Return	Description
A T T R I B U T E S	-	first_name	str	Stored separately from last_name for flexible display and sorting.
	-	last_name	str	Stored separately from first_name .
	-	email	str	Unique login identifier. Validated for format and uniqueness before saving.
	-	hashed_password	str	The password is never stored in plain text. Hashed using bcrypt before storage.
	-	is_admin	bool default = False	Boolean flag. Reserved for future admin features.
	+	age	int	Used to store user's age and filter recommendations by age rating. Not required at registration.
M E T H O D S	+	username()	str	Computed property returning first_name + last_name concatenated. Not stored in the database.
	+	validate_name()	None	Checks that first and last names are non-empty strings within defined character limits.
	+	validate_email()	None	Checks email format and uniqueness in the database. Raises an error if already taken.
	+	validate_password()	None	Checks password complexity: minimum length and required character types.
	+	hash_password()	None	Transforms the plain-text password into a bcrypt hash. Called before save() on creation or password update.
	+	verify_password()	bool	Compares a plain-text input against the stored bcrypt hash. Returns True if valid. Used at login.
	+	get_watchlist()	List [WatchlistEntry]	Queries the database and returns all WatchlistEntry records for this user.
	+	get_viewing_	List [ViewingHistory]	Queries the database and returns all ViewingHistoryEntry records for this user.

	Visibility	Name	Type / Return	Description
		history()	Entry]	
	+	get_platforms()	List [Platform]	Returns all Platform objects linked to this user via the join table.

Note: A 'preferred_language' attribute (VARCHAR, default 'en') may be added post-MVP to support language preferences.

Film <<DTO>>

A Data Transfer Object populated from the TMDB API response. It is never saved to the database - it is a temporary container used to carry film data between the backend and the frontend.

	Visibility	Name	Type	Description
A T T R I B U T E S	+	tmdb_id	int	TMDB unique identifier.
	+	title	str	Film title, returned in English (en-US) for the MVP. French language support (fr-FR) is planned as a post-MVP feature.
	+	year	int	Release year.
	+	genres	list[str]	List of genre labels from TMDB.
	+	poster_url	str	Full URL to the film poster image.
	+	synopsis	str	Short plot description in the requested language.
	+	director	str	Director name, extracted from TMDB crew data.
	+	cast	list[str]	List of main cast member names.
	+	runtime	int	Film duration in minutes.
	+	streaming_platforms	list	Platforms where the film is available, from TMDB Watch Providers.

Film has no methods - it is a passive data container.

WatchlistEntry <<entity>>

Represents a single film saved in a user's watchlist. References the film by **tmdb_id** only - no metadata is stored locally.

	Visibility	Name	Type / Return	Description
A T T R I B U T E S	-	user_id	UUID4	Foreign key linking the entry to its owner. Never exposed to the frontend.
	+	tmdb_id	int	TMDB identifier of the film.
M E T H O D S	+	add()	None	Saves the entry to the database.
	+	remove()	None	Deletes the entry from the database.
	+	mark_as_watched()	ViewingHistoryEntry	Creates a new ViewingHistoryEntry from this entry, then deletes this WatchlistEntry.
	+	get_film_details()	Film	Calls the TMDB API with tmdb_id and returns a fully populated Film DTO.

ViewingHistoryEntry <<entity>>

The core entity of CinéMood. Stores the user's personal experience of a film through tags, a prestige tier, and an optional personal note. The **created_at** field inherited from BaseModel serves as the watch date.

	Visibility	Name	Type / Return	Description
A T T R I B U T E S	-	user_id	UUID4	Foreign key linking the entry to its owner. Never exposed to the frontend.
	+	tmdb_id	int	TMDB identifier of the film.
	+	tags	list[Tag]	Tags associated with this entry, managed via the viewing_history_tags join table.
	+	prestige_tier	PrestigeTier	Optional global rating using the PrestigeTier enumeration.
	+	personal_note	str	Optional free-text personal review.
M E T H O D S	+	add_tag()	None	Inserts a row in the viewing_history_tags join table.
	+	remove_tag()	None	Deletes the corresponding row from the join table.
	+	update_prestige_tier()	None	Updates the prestige_tier field and calls save().
	+	get_film_details()	Film	Calls the TMDB API with tmdb_id and returns a fully populated Film DTO.

Tag <<entity>>

A predefined emotional or contextual label. Tags are fixed and shared across all users.
Managed by the application at initialisation.

	Visibility	Name	Type / Return	Description
A T T R I B U T E S	+	id	int	Integer primary key, auto-incremented by PostgreSQL.
	+	name	str	Tag label displayed to the user (e.g. "Guilty pleasure").
	+	description	str	Short explanation displayed as a tooltip in the UI.

Tag has no methods - its records are managed directly by the application at initialisation.

Platform <<entity>>

A streaming platform available to users. The list is fixed and managed by the application. Used to filter recommendations based on the user's subscriptions.

	Visibility	Name	Type	Description
A T T R I B U T E S	+	id	int	Integer primary key, auto-incremented by PostgreSQL.
	+	name	str	Platform name (e.g. "Netflix", "Prime Video").
	+	logo_url	str	URL to the platform logo, displayed in the UI.

Platform has no methods - its records are managed directly by the application at initialisation.

PrestigeTier <<enumeration>>

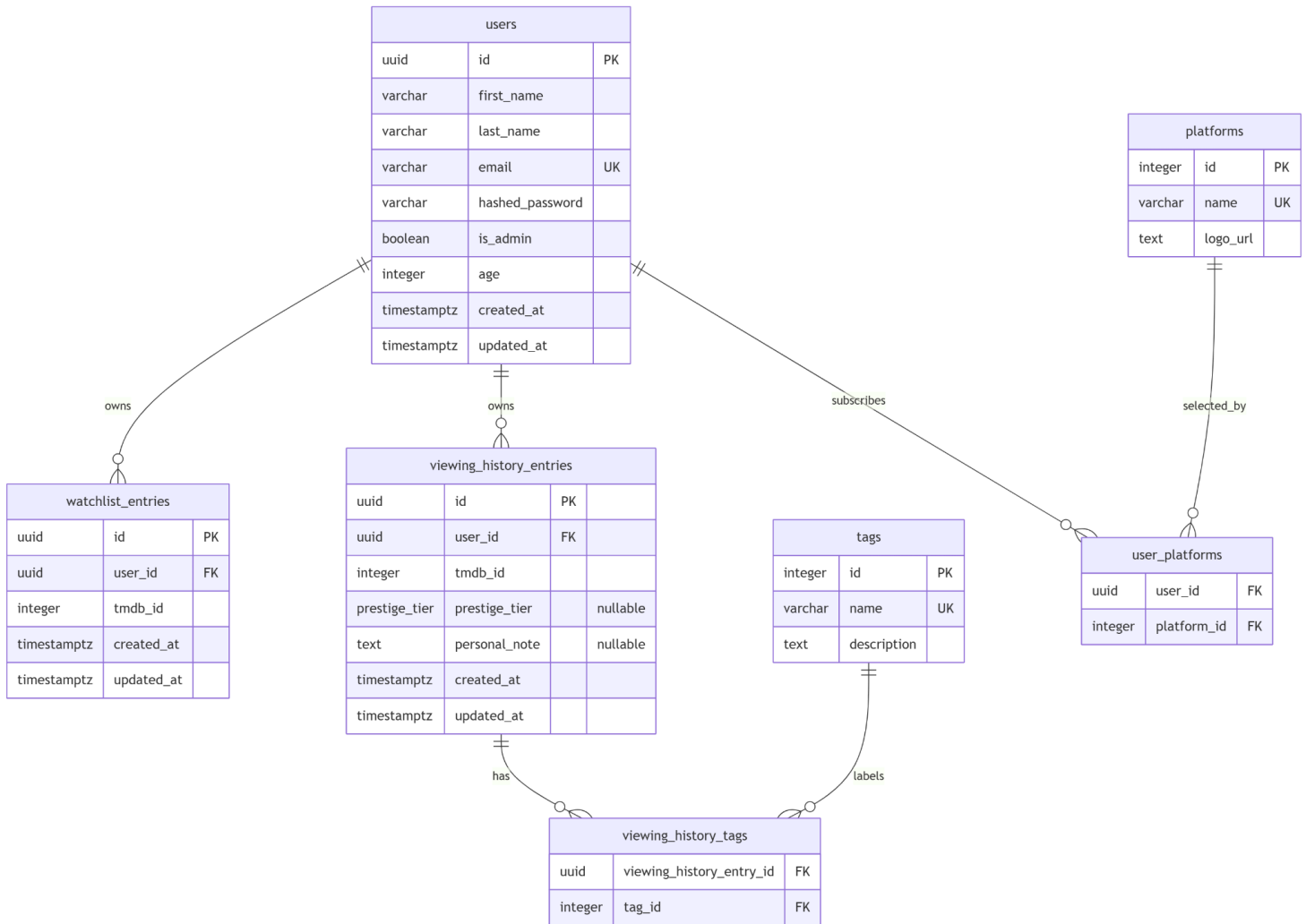
A fixed set of values representing the user's global rating of a film.

Value	Intended meaning
PLATINUM	An absolute favourite - a personal masterpiece
GOLD	Excellent - highly recommended
SILVER	Good - worth watching
BRONZE	Average - watchable but unremarkable
TRASH	Poor - would not recommend

6.2. ER Diagram

The database is designed for PostgreSQL and stores only the application's persistent data: users, watchlist entries, viewing history entries, tags, platforms, and relationship tables.

Film metadata is not stored locally because CinéMood uses the TMDB API as the source of truth. Only the `tmdb_id` is stored in the database when a user saves or watches a film.



Relationship Summary

Relationship	Entity A	Entity B	Type	Join table
A user owns watchlist entries	users	watchlist_entries	One-to-many	
A user owns viewing history entries	users	viewing_history_entries	One-to-many	
A user subscribes to platforms	users	platforms	Many-to-many	user_platforms

HOLBERTON SCHOOL BORDEAUX

A viewing history entry is labeled with tags	Viewing_ history_entries	tags	Many-to-many	viewing_ history_tags
--	-----------------------------	------	--------------	--------------------------

7. Endpoints and URLs

Following is a complete list of the application's API endpoints, both internal and external.

7.1. External Endpoints

External: TMDB

Endpoint	Method	Usage	Example
/search/movie	GET	Search films by title	/search/movie?query=inception&language=en-US
/movie/{tmdb_id}	GET	Get full film details	/movie/27205?append_to_response=credits
/movie/{tmdb_id}/watch/providers	GET	Get streaming availability	/movie/27205/watch/providers

Base URL : **https://api.themoviedb.org/3**

Auth : Bearer token in the header - **Authorization: Bearer {TMDB_API_KEY}**

External: Mistral AI

Endpoint	Method	Usage
/v1/chat/completions	POST	Send prompt, receive film suggestions

Base URL : **https://api.mistral.ai**

Auth : Bearer token in the header - **Authorization: Bearer {MISTRAL_API_KEY}**

Example system prompt structure: The following example illustrates how a prompt is structured when sent to the LLM. The exact wording will be refined iteratively during development, but the structure and output format are fixed.

```
System:
You are a film recommendation assistant for CinéMood.
You must only suggest films - never series, documentaries, or other
content.
Always respond in JSON format only, with no additional text:
[{"tmdb_id": 0, "title": "", "year": 0, "reason": ""}]
Never suggest films already present in the user's viewing history.
Keep reasons short, personal, and encouraging (2 sentences max).

User:
The user has watched: Inception (2010), The Dark Knight (2008),
Interstellar (2014).
Current mood: tired, wants something light and fun.
Available platforms: Netflix France, Prime Video France.
Suggest 3 films.
```

Note: This prompt is indicative and will be refined during development through iterative testing. The system prompt may evolve to incorporate swipe results, age preferences, and other contextual signals as the recommendation engine matures.

7.2. Internal Endpoints

Internal: Auth endpoints (/auth/*)

Path	Method	Input	Output	Usage
/auth/register	POST	{first_name, last_name, email, password}	201 {jwt_token, user_id}	Create user account
/auth/login	POST	{email, password}	200 {jwt_token, user_id}	Authenticate and get JWT

Internal: User endpoints (/user/*)

Path	Method	Input	Output	Usage
/users/me	GET	JWT header	200 {user profile}	Get current user profile
/users/me	PATCH	{first_name?, last_name?, age?}	200 {updated user}	Update user profile
/users/me/ platforms	GET	JWT header	200 [{platform}]	Get user platform preferences
/users/me/ platforms	PUT	{platform_ids[]}	200 {updated platforms}	Set user platform preferences

Internal: Film endpoints (/films/*)

Path	Method	Input	Output	Usage
/films/search	GET	?query=title	200 [{film cards}]	Search films via TMDB
/films/{tmdb_id}	GET	JWT header	200 {Film DTO + status}	Get film details + user status
/films/log	POST	{tmdb_id, tag_ids[]}	201 {entry_id}	Log a watched film
/films/log/{entry_id}	DELETE	JWT header	200 {}	Remove film from history
/films/history	GET	JWT header	200 [{entries}]	Get viewing history
/films/watchlist	POST	{tmdb_id}	201 {entry_id}	Add film to watchlist
/films/watchlist/{tmdb_id}	DELETE	JWT header	200 {}	Remove from watchlist
/films/watchlist	GET	JWT header	200 [{entries}]	Get watchlist

Internal: Recommendation endpoints (/recommendations /*)

Path	Method	Input	Output	Usage
/recommendations/start	POST	{mood, swipe_results[], optional_prompt?}	200 [{film, platform, reason}]	Get film recommendations

8. Frontend Components and Design

The frontend of CinéMood is designed as a **React-based web application**. Its role is to translate the core user stories into a clear, fluid, and interactive user experience, while communicating with the **FastAPI backend** through **REST API calls**.

The interface is built around **reusable components** such as “film cards”, “dashboard sections”, “tag selectors”, “platform selectors”, “mood questionnaire screens”, “swipe cards”, and “recommendation result cards”. This component-based approach makes the frontend easier to maintain, avoids duplication, and allows the same visual elements to be reused across different parts of the application.

The frontend is structured around several main views, such as the dashboard, **film catalog**, **recommendation experience**, and **user profile**.

Navigation between these views can be handled with **React Router** or an equivalent routing solution, depending on the final implementation.

Tailwind CSS is used to style the interface directly inside React components through utility classes. This choice supports rapid UI development, responsive design, and a custom visual identity adapted to CinéMood's cinematic atmosphere.

The frontend does not communicate directly with the database or external APIs such as TMDB or Mistral AI.

All data exchanges are routed **through the backend**, which handles authentication, business logic, API keys, database access, and external service calls.

This separation improves security and keeps the frontend focused on presentation, interaction, and user experience.

8.1. React Frontend Architecture

The React frontend is designed around views or pages, shared components, routing, and a centralized API communication layer.

Pages represent complete screens of the application. Each page is connected to one or several user stories and is responsible for managing screen-level state, triggering API requests, and composing reusable UI components.

Shared components represent smaller reusable interface elements, such as film cards, buttons, tag chips, platform badges, forms, and recommendation cards. They receive data through props and communicate user actions back to their parent page through callback functions.

A centralized API client is planned to group HTTP requests to the FastAPI backend. This prevents duplicated fetch logic across components and ensures that authentication headers, error handling, and response parsing are managed consistently.

The frontend follows the following general structure:

- **Pages:** DashboardPage, CatalogPage, FilmLogPage, RecommendationPage, ProfilePage.
- **Shared UI components:** FilmCard, TagSelector, PlatformSelector, MoodQuestionnaire, SwipeDeck, RecommendationCard.
- **State management:** local React state for UI interactions, global authentication state for logged-in user data and JWT token.
- **API communication:** all requests are sent to the backend REST API, never directly to PostgreSQL or external services.

The app is designed to be **responsive** from the MVP development stage: it must display correctly on **both desktop and mobile screens**, and the user experience must **not be affected by the device** being used.

8.2. React Components

The following table lists the main React components planned for the CinéMood frontend.

Components are divided into two categories:

- Pages (full-screen views connected to user stories)
- Shared UI Components (reusable building blocks used across multiple pages)

Pages

Component	Connected User Stories	Description
AuthPage	US-01, US-02	Entry point for unauthenticated users. Contains the login and registration forms. Redirects to DashboardPage on successful authentication.
DashboardPage	US-05	Main screen after login. Displays the user's recent viewing history, watchlist, and the recommendation entry point. Composes FilmCard , WatchlistSection , and HistorySection .
CatalogPage	US-03	Film search interface. Contains the search bar and the results grid. Triggers API calls to <code>/films/search</code> and renders FilmCard components for each result.
FilmDetailPage	US-04, US-06, US-15	Displays full film details (poster, synopsis, cast, genres, runtime). Fetches data from <code>/films/{tmdb_id}</code> . Renders action buttons (" Log this film " / " Remove from history ", " Add to watchlist " / " Remove from watchlist ") based on the film's current status in the user's data.
Recommendation Page	US-07, US-08, US-09, US-11	Hosts the full recommendation experience: mood questionnaire , swipe session , and results display . Manages the recommendation flow state locally and submits to <code>/recommendations/start</code> on completion.
ProfilePage	US-10, US-14	Displays and allows editing of the user's profile (name , age) and streaming platform preferences . Communicates with <code>/users/me</code> and <code>/users/me/platforms</code> .

Shared UI Components

Component	Props	Description
FilmCard	<code>tmdb_id,</code> <code>title, year,</code> <code>poster_url,</code> <code>genres,</code> <code>status?</code>	Displays a compact film card with poster, title, year, and genre badges. Used in search results, watchlist, viewing history, and recommendation results. Accepts an optional status to adapt its action buttons.
TagSelector	<code>tags[],</code> <code>selectedTags[]</code> <code>, onChange</code>	Displays the list of available tags as chips. Allows multi-selection. Used in the film logging flow when the user selects tags before saving a viewing history entry.
Platform Selector	<code>platforms[],</code> <code>selectedPlatforms[],</code> <code>onChange</code>	Displays the list of available streaming platforms as logo chips. Used in the ProfilePage and as part of the registration flow (US-10).
Mood Questionnaire	<code>onComplete(moodData)</code>	Multi-step questionnaire presented at the start of the recommendation experience. Handles its own step state internally and calls <code>onComplete</code> when the user finishes. Data is passed to RecommendationPage, not sent to the backend directly.
SwipeDeck	<code>films[],</code> <code>onSwipeComplete(results[])</code>	Renders a stack of SwipeCard child components. Handles swipe gesture detection (right = interested, left = not interested). Accumulates results in local state and calls <code>onSwipeComplete</code> when the deck is exhausted.
SwipeCard	<code>film, onSwipe</code> <code>(direction)</code>	Displays a single film card in the swipe session (poster, synopsis, tags/badges). Reports the swipe direction to SwipeDeck via <code>onSwipe</code> .
Recommendation Card	<code>film, reason,</code> <code>platform</code>	Displays a single recommendation result with the film poster, the LLM-generated reason, and the streaming platform logo. Includes an "Add to watchlist" button.
NavBar	<code>currentUser</code>	Persistent top navigation bar. Displays the app logo, navigation links (Dashboard, Catalog, Recommendation, Profile), and a logout button. Shown on all authenticated pages.
LoadingSpinner	<code>message?</code>	Generic loading indicator used while API calls are in progress.
ErrorMessage	<code>message,</code> <code>onRetry?</code>	Displays error messages from failed API calls with an optional retry button.

8.3. Frontend Interactions

This section describes the key interactions between the React components and the backend API, organized by user flow.

Authentication Flow

When the application loads, it checks for a **JWT token** in memory (React Context). If no token is found, the user is redirected to **AuthPage**.

On the registration form, the user fills in first name, last name, email, and password. On submission, **AuthPage** sends a **POST** request to **/auth/register**. On success (**201 Created**), the returned **JWT token** is stored in the global authentication context and the user is redirected to **DashboardPage**.

On the login form, **AuthPage** sends a **POST** request to **/auth/login**. On success (**200 OK**), the **JWT token** is stored and the user is redirected to **DashboardPage**.

The **JWT token** is attached to every subsequent API request as a **Bearer token** in the **Authorization header**. The centralized API client handles this automatically, so individual components do not need to manage authentication headers directly.

Dashboard and Film Catalog

When **DashboardPage** mounts, it sends **two concurrent GET** requests to **/films/history** and **/films/watchlist**. The responses populate two separate sections: **viewing history** and **watchlist**.

Each section renders a list of **FilmCard** components. Clicking a **FilmCard** navigates to **FilmDetailPage** for that film's **tmdb_id**.

On **CatalogPage**, the user types a query in the search bar and submits. A **GET** request is sent to **/films/search?query=...**. Results are rendered as a **grid of FilmCard** components. Clicking a card navigates to **FilmDetailPage**.

Film Detail and Logging

When **FilmDetailPage** mounts for a given **tmdb_id**, a **GET** request is sent to **/films/{tmdb_id}**. The response includes both the **Film DTO** (details from TMDB) and a **status object** indicating whether the film is already in the user's viewing history or watchlist.

The action buttons adapt immediately based on this status:

- If **in_viewing_history** is false, the button shows "**Log this film**".
- If **in_viewing_history** is true, the button shows "**Remove from history**".
- If **in_watchlist** is false, the button shows "**Add to watchlist**".
- If **in_watchlist** is true, the button shows "**Remove from watchlist**".

When the user clicks "**Log this film**", a tag selection modal opens with the **TagSelector** component. After selecting tags and confirming, a **POST** request is sent to **/films/log** with the **tmdb_id** and the **selected tag_ids[]**. On success (**201 Created**), the local status state is updated - no page reload is needed. The button switches immediately to "**Remove from history**" using React's **useState**.

When the user clicks "**Remove from history**", a **DELETE** request is sent to **/films/log/{entry_id}**. On success (**200 OK**), the button switches back to "**Log this film**".

When the user clicks "**Add to watchlist**", a **POST** request is sent to **/films/watchlist** with the **tmdb_id**. On success, the button switches to "**Remove from watchlist**".

Recommendation Experience

The recommendation experience is managed entirely within **RecommendationPage** and its **child components**. No intermediate data is sent to the backend until the final submission.

- **Step 1 - Mood Questionnaire:** **MoodQuestionnaire** renders a multi-step form. The user answers questions about their current mood and preferences. On completion, the mood data is stored in **RecommendationPage's local state**.
- **Step 2 - Swipe Session:** **SwipeDeck** receives an initial set of film cards (fetched from the backend based on the mood data, or pre-generated by the LLM). The user swipes right (interested) or left (not interested). **SwipeDeck** accumulates the results in **local state**. No API call is made during swiping.
- **Step 3 - Optional prompt:** Before submitting, the user may add a **free-text prompt** for additional context.

- **Step 4 - Submission:** **RecommendationPage** sends a **POST** request to **/recommendations/start** with the **mood** object, **swipe_results[]** array, and **optional_prompt**.
The backend processes the request and returns a list of enriched recommendations.
- **Step 5 - Results:** The response is rendered as a list of **RecommendationCard components**, each showing the film, the **LLM-generated** reason, and the **platform availability**. The user can add any film to their watchlist directly from the results.

During the swipe session, the **NavBar** is hidden to provide an immersive, distraction-free experience. It reappears on the results screen.

Profile Management

On **ProfilePage**, the user can **update** their **name** and age via a **PATCH** request to **/users/me**, and **update** their **streaming platform subscriptions** via a **PUT** request to **/users/me/platforms** with an array of **platform_ids[]**.

PlatformSelector displays the available platforms as clickable logo chips. The selected state is managed locally and submitted as a batch on save.

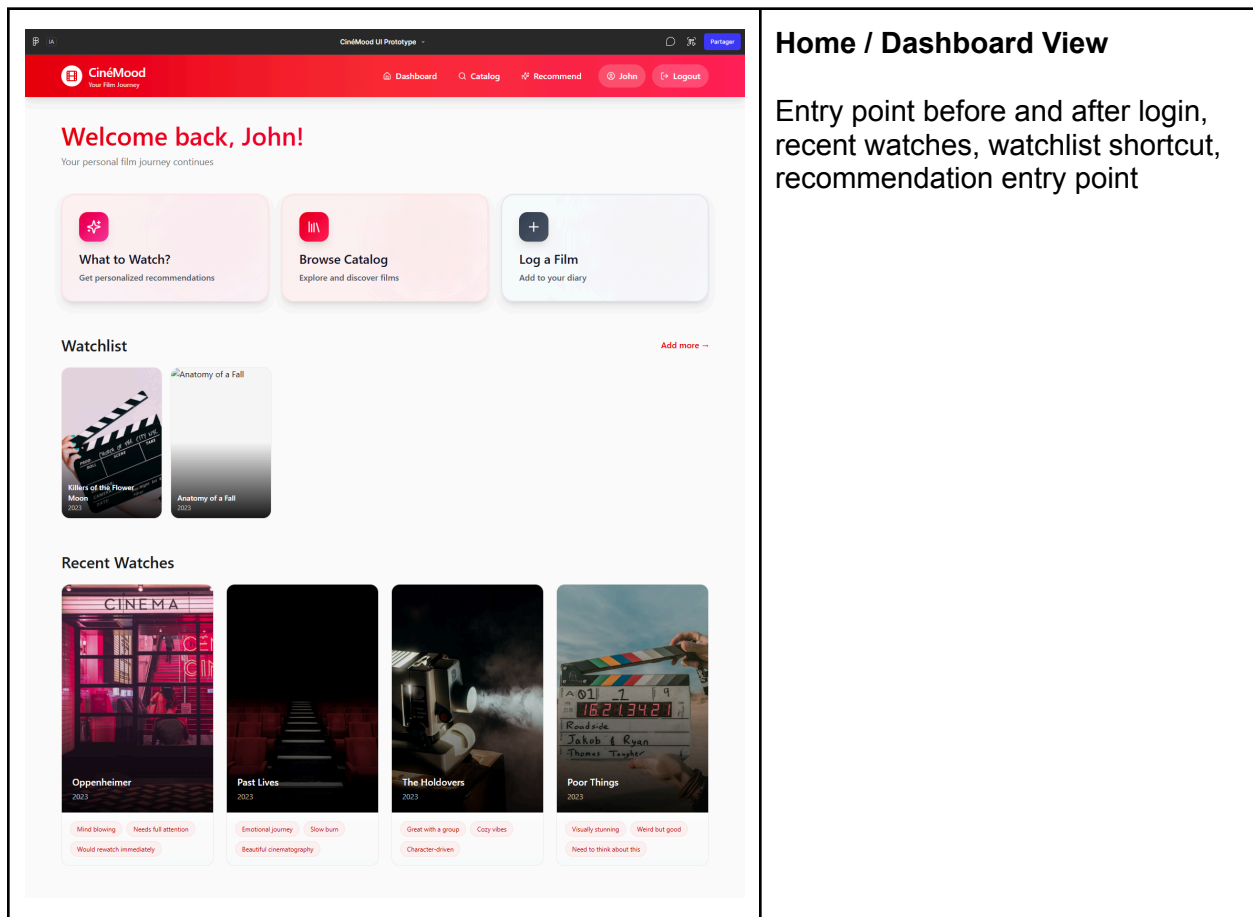
State Management Summary

State type	Managed by	Tool
JWT token + user info	Global	React Context (AuthContext)
Screen-level data (film lists, search results)	Page components	useState / useEffect
Swipe session results	SwipeDeck	useState (local)
Mood questionnaire progress	MoodQuestionnaire	useState (local)
Tag selection	TagSelector	useState (local, lifted to FilmDetailPage)
Platform selection	PlatformSelector	useState (local, lifted to ProfilePage)

8.4. Mockup

A complete navigable and interactive mockup prototype of the app is available at this [link](#).

These mockups are not final UI specifications. Their purpose is to validate the main user journey, the screen hierarchy, and the reusable interface components planned for the MVP.



CinéMood

Your personal film diary & recommendation engine

Log In

Sign Up

Email

Password

Log In

CinéMood

Your personal film diary & recommendation engine

Log In

Sign Up

First Name

Last Name

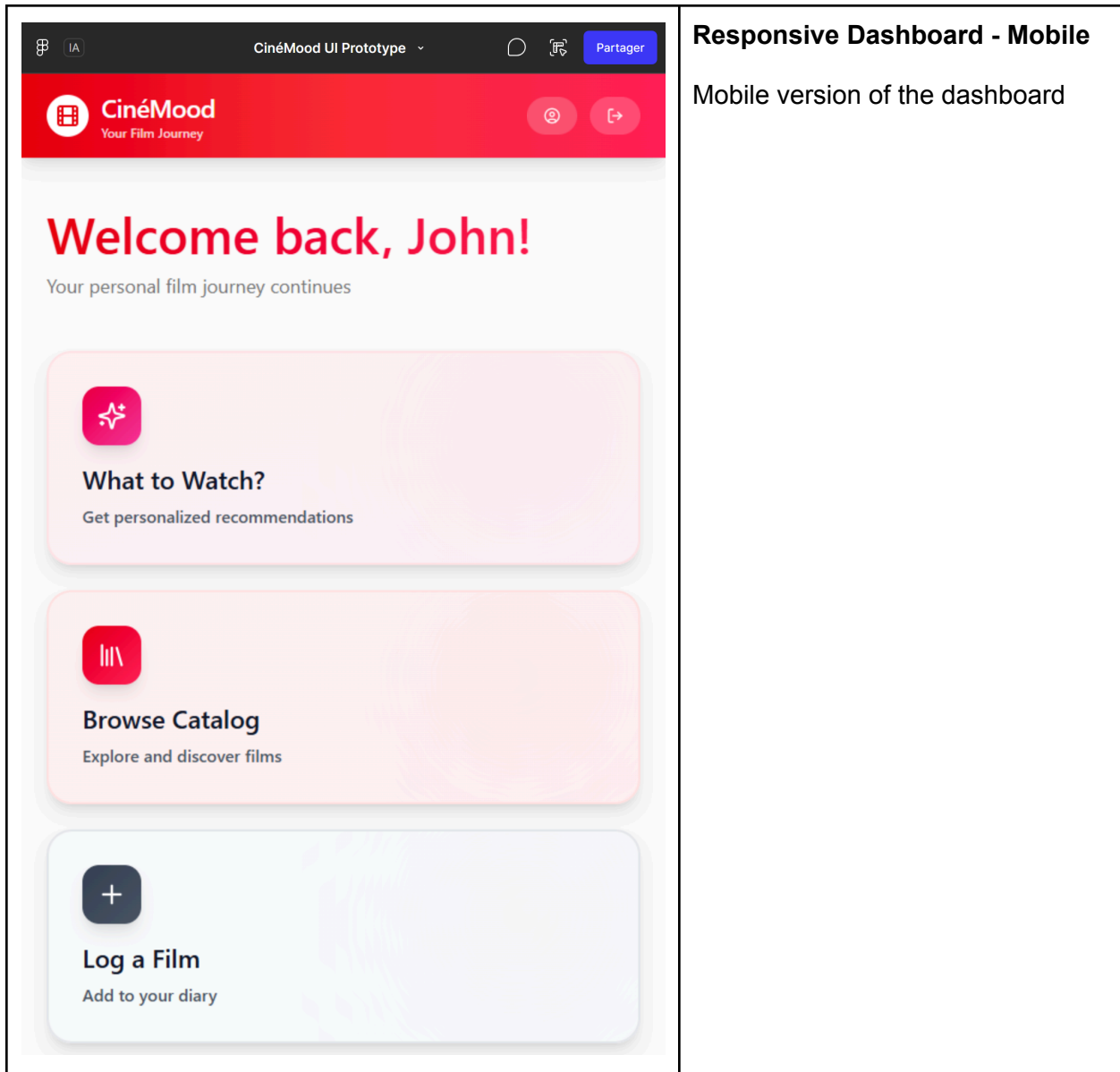
Email

Password

Create Account

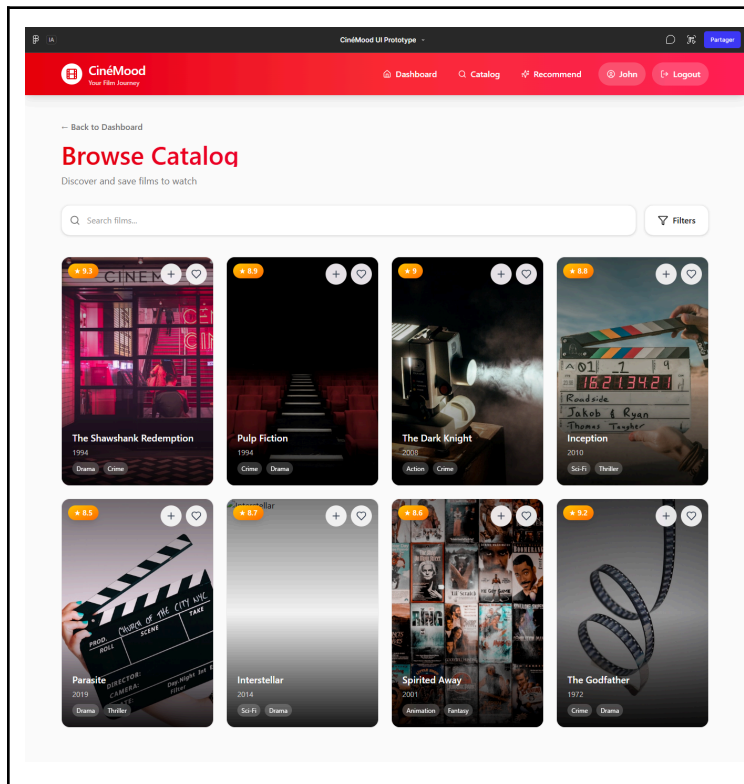
User Registration / Login View

Entry point before login, recent watches, watchlist shortcut, recommendation entry point



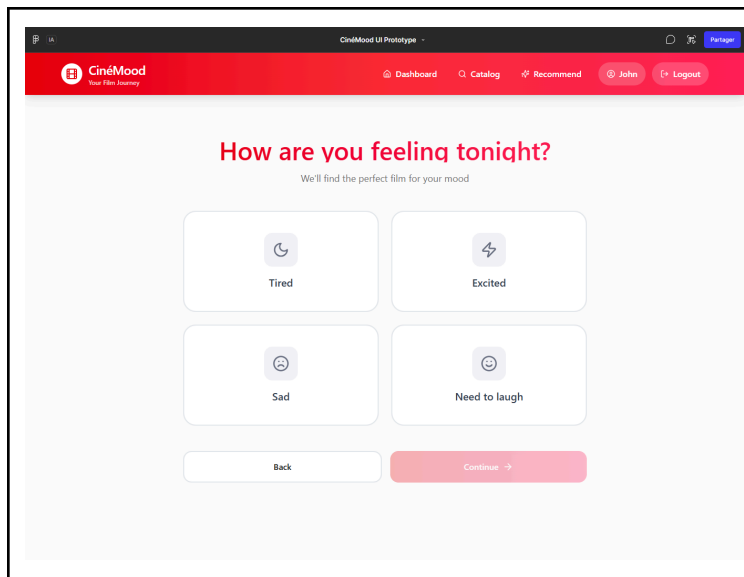
Responsive Dashboard - Mobile

Mobile version of the dashboard



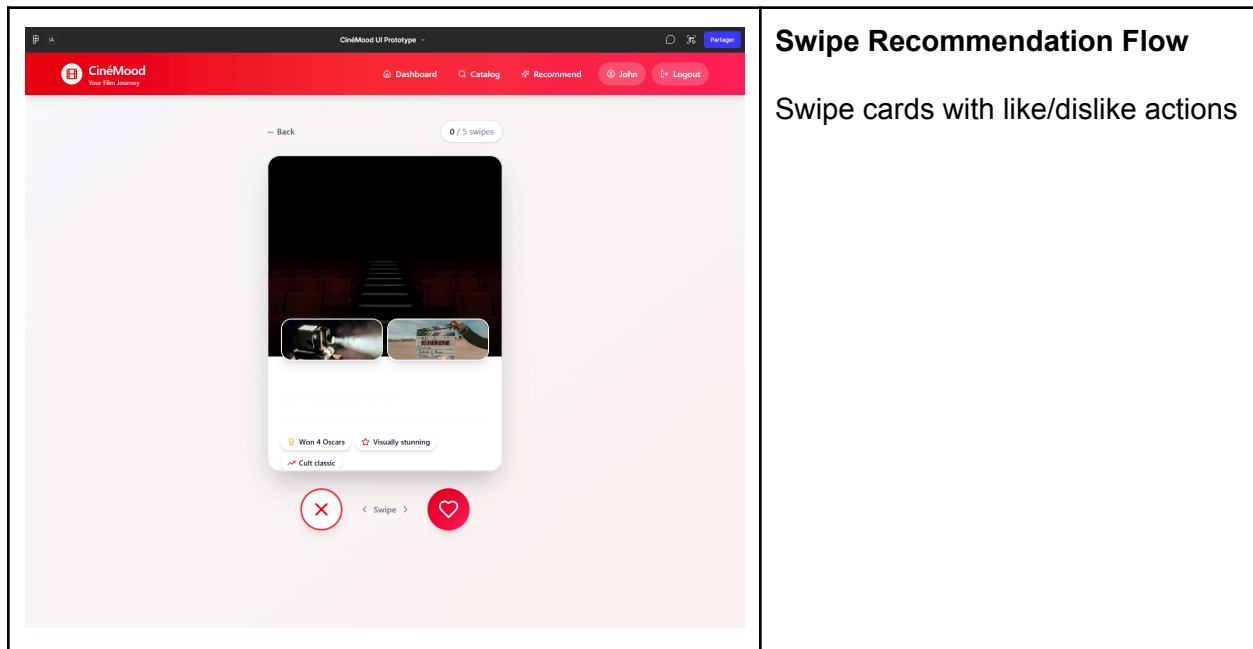
Movie Catalog View

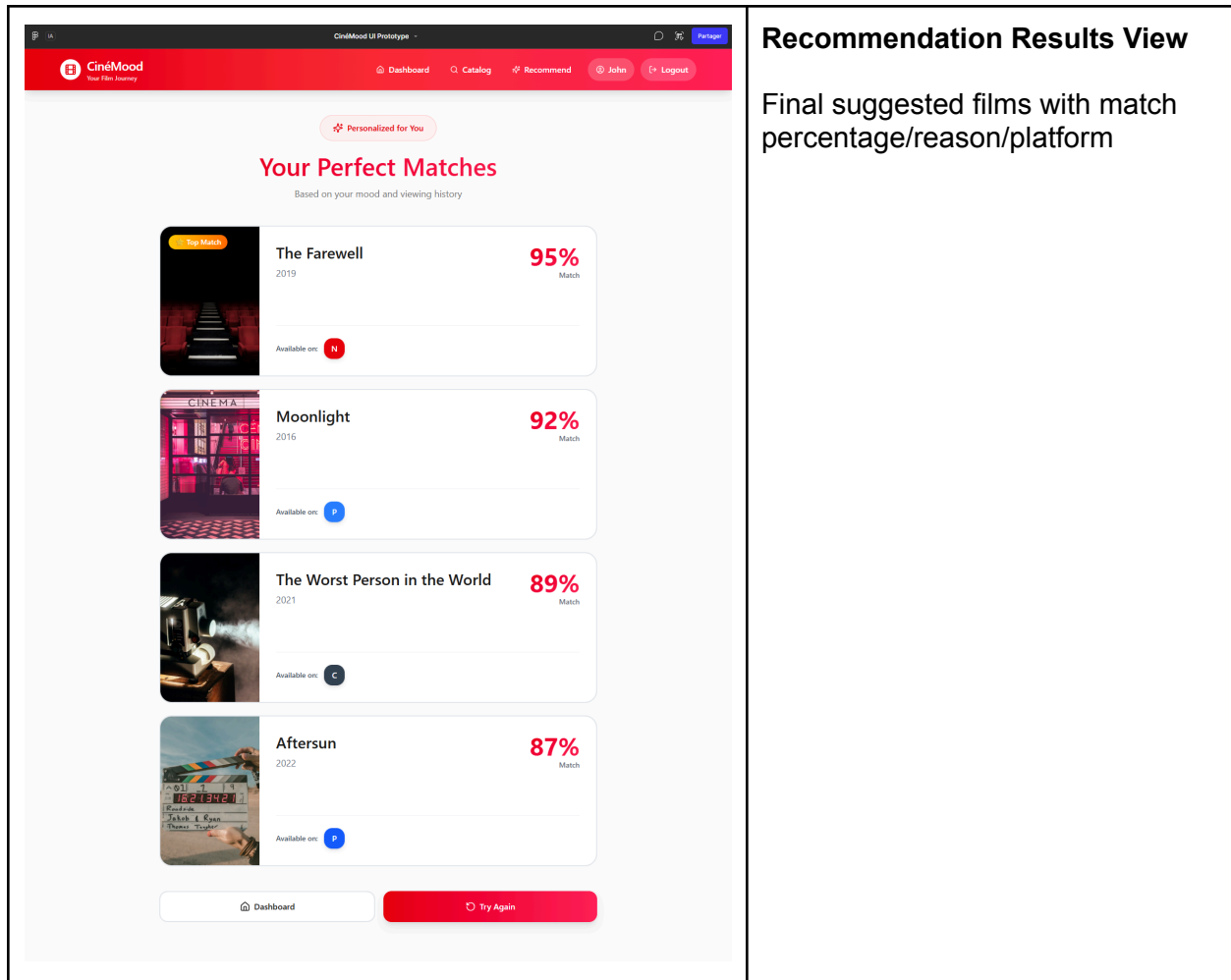
Film search or browsing interface with film cards.



Mood Questionnaire View

First step of the recommendation experience





9. Sequence Diagrams

This section illustrates the key interaction flows of the CinéMood application. Each diagram shows how the system components communicate step by step for a given use case, covering both the happy path and the main error cases.

Five critical flows are documented below, ordered by complexity - from user authentication to the full recommendation experience.

Diagram	Use case	Components involved
1	User Registration	Frontend, Backend, User Service, PostgreSQL
2	User Login	Frontend, Backend, User Service, PostgreSQL
3	Searching for a Movie	Frontend, Backend, Film Service, TMDB API
4	Log a Film	Frontend, Backend, Film Service, User Service, TMDB API, PostgreSQL
5	Film Recommendation Experience	Frontend, Backend, Recommendation Facade, User Service, PostgreSQL, TMDB API, Mistral AI

9.1. User Registration

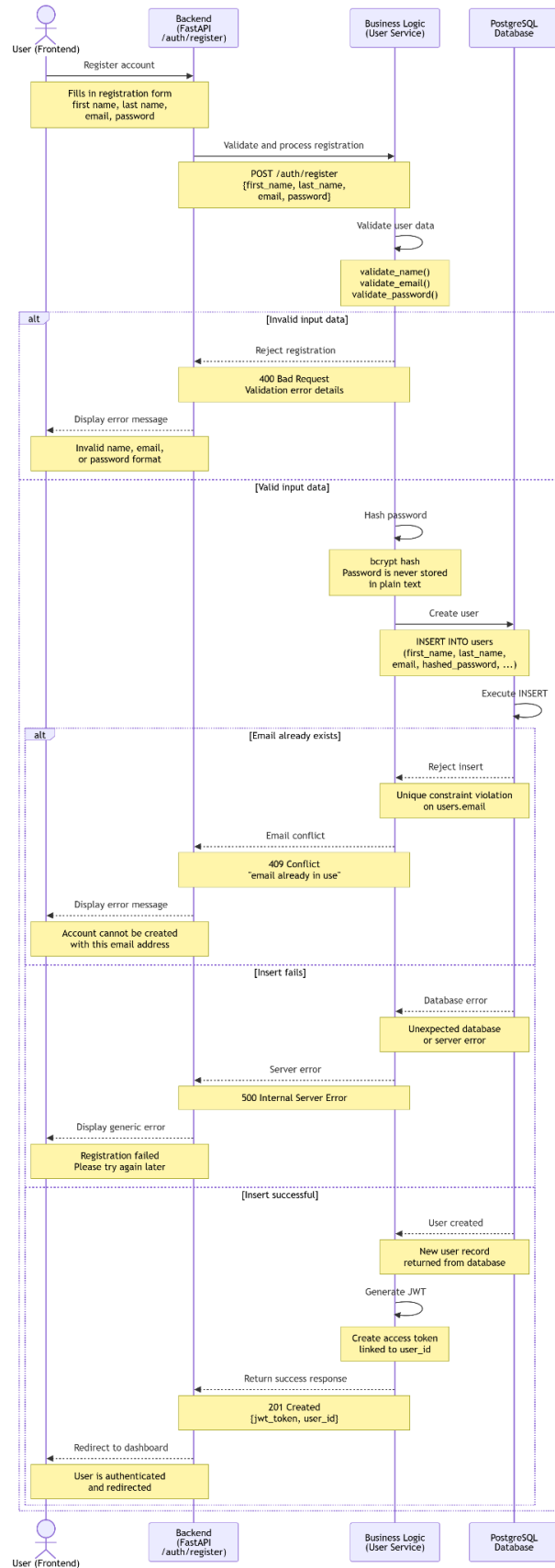
This sequence covers the full flow when a new user creates an account on CinéMood.

Actors:

- **User** - user interacting through the React frontend
- **Backend** - the /auth/register route of the FastAPI
- **Business Logic** - the User Service of the FastAPI
- **Database** - the PostgreSQL database

Key steps:

- 1) The user submits the registration form via the frontend
- 2) The frontend sends a POST request to /auth/register
- 3) The backend routes the request to the User Service
- 4) The User Service validates the input data (name, email, password format)
- 5) The User Service hashes the password using bcrypt
- 6) The User Service inserts the new user record into the database
- 7) If the email already exists, a 409 Conflict is returned
- 8) If the insert fails, a 500 Internal Server Error is returned
- 9) If successful, a JWT token is generated and returned with a 201 Created response
- 10) The frontend redirects the authenticated user to the dashboard



9.2. User Login

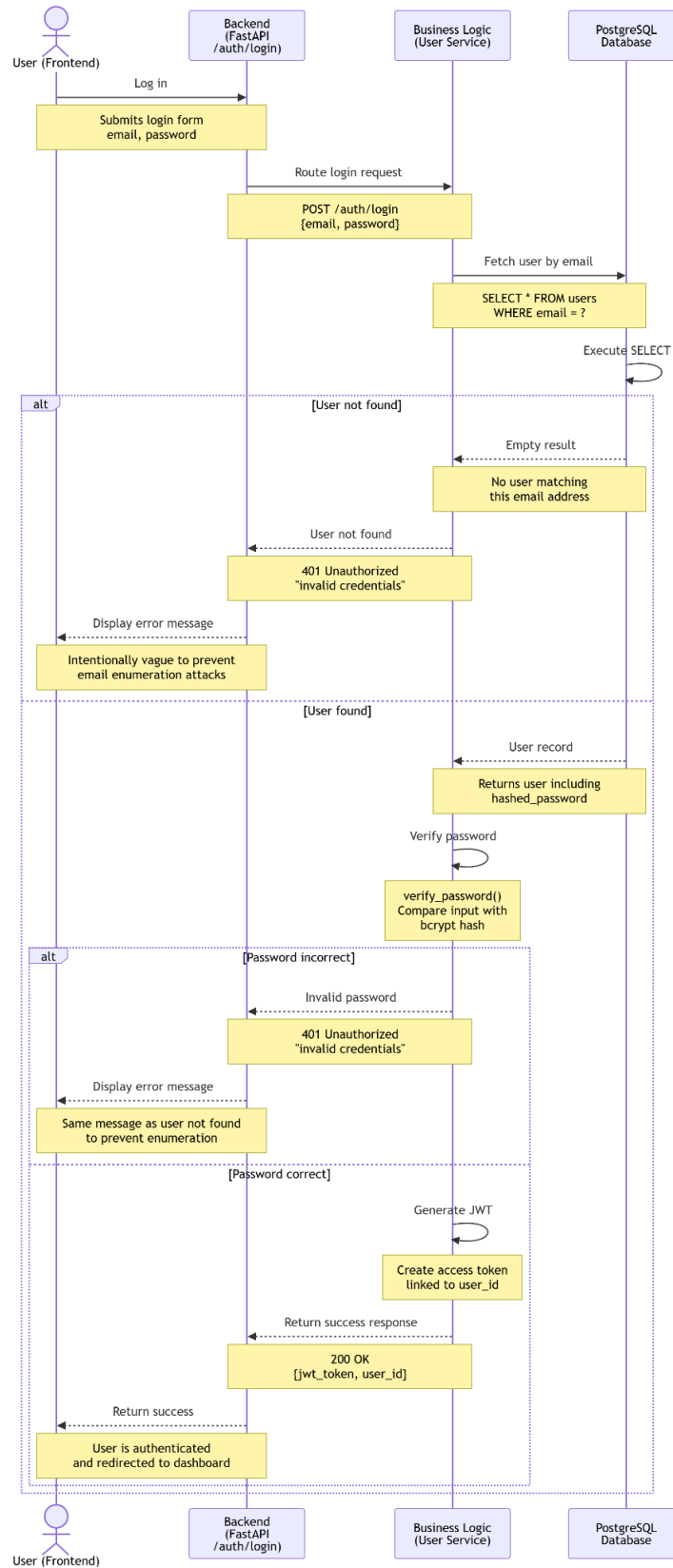
This sequence covers the flow when a returning user logs into their account.

Actors:

- **User** - user interacting through the React frontend
- **Backend** - the `/auth/login` route of the FastAPI
- **Business Logic** - the User Service of the FastAPI
- **Database** - the PostgreSQL database

Key steps:

- 1) The user submits their email and password via the frontend
- 2) The frontend sends a POST request to `/auth/login`
- 3) The backend routes the request to the User Service
- 4) The User Service queries the database for the user by email
- 5) If the user is not found, a 401 Unauthorized is returned
- 6) The User Service verifies the password against the stored bcrypt hash
- 7) If the password is incorrect, a 401 Unauthorized is returned
- 8) If valid, a JWT token is generated and returned with a 200 OK response
- 9) The frontend stores the token and redirects to the dashboard



9.3. Searching for a Movie

This sequence covers the flow when a user searches for a movie in the TMDB catalog, in order to log it or add it to their watchlist.

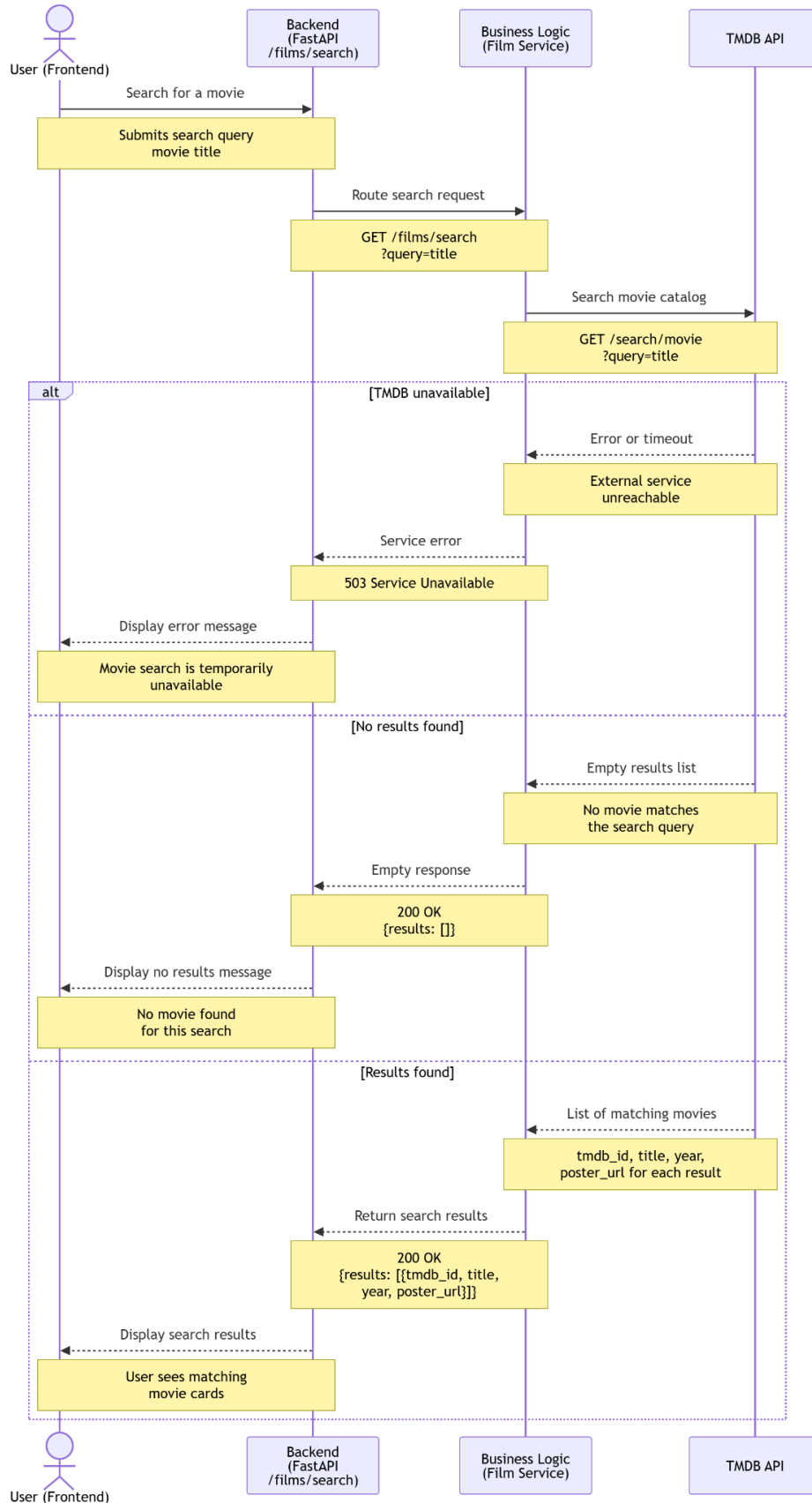
Actors:

- **User** - user interacting through the React frontend
- **Backend** - the /films/search route of the FastAPI
- **Business Logic** - the Film Service of the FastAPI
- **API** - the TMDB API

Key steps:

- 1) The user navigates through the app to the movie search page
- 2) The user types a movie title in the search bar and confirms
- 3) The frontend sends a GET request to /films/search?query=...
- 4) The backend routes the request to the Film Service
- 5) The Film Service calls the TMDB search endpoint
- 6) TMDB returns a list of matching films (multilingual search by default)
- 7) The backend returns the results to the frontend
- 8) The user selects a film from the results

HOLBERTON SCHOOL BORDEAUX



9.4. Log a Film

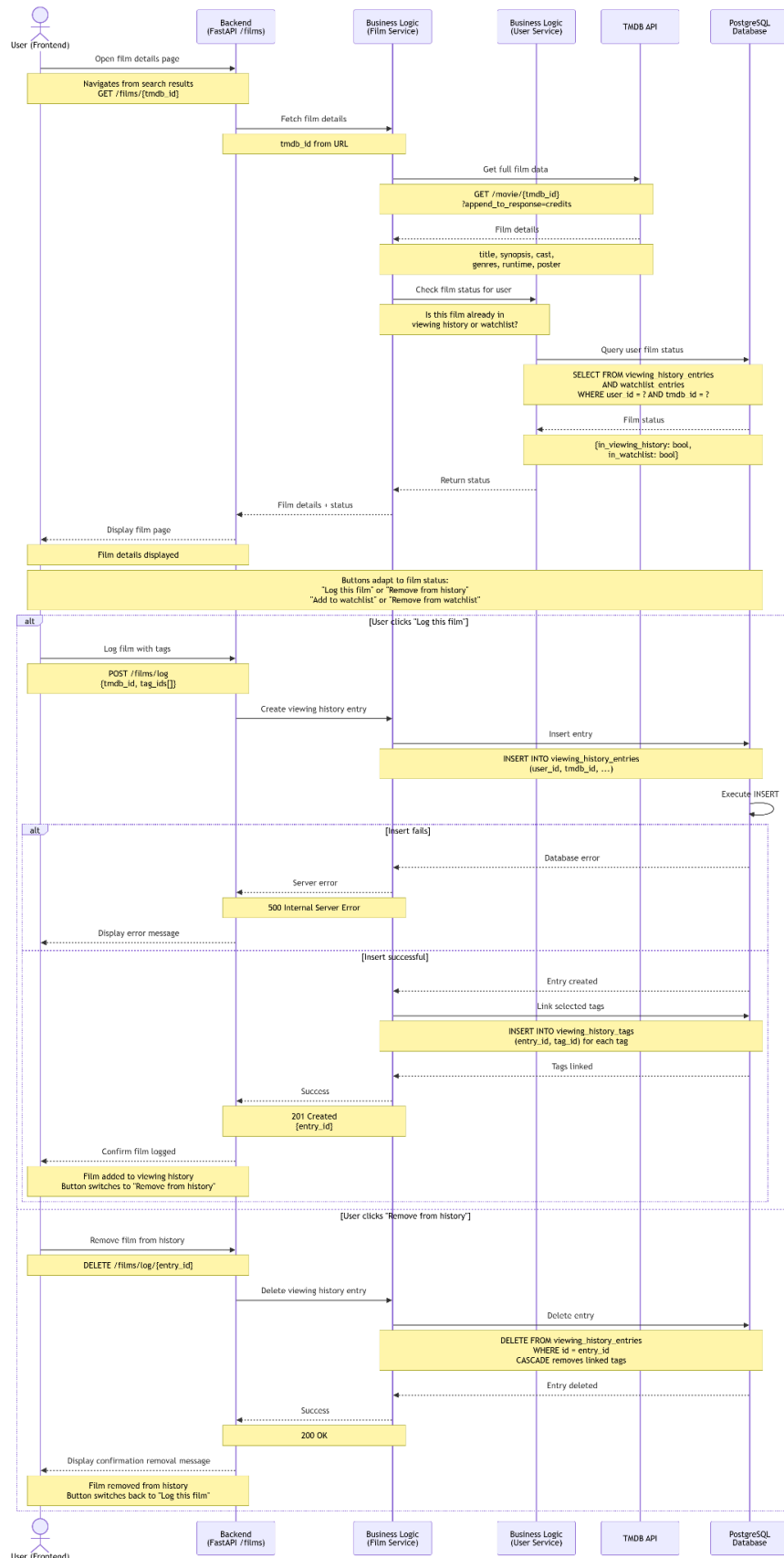
This sequence covers the flow when a user views a film's details page and logs it to their viewing history.

Actors:

- **User** - user interacting through the React frontend
- **Backend** - the `/films/*` routes of the FastAPI
- **Business Logic - Film Service** - the Film Service of the FastAPI
- **Business Logic - User Service** - the User Service of the FastAPI
- **API** - the TMDB API
- **Database** - the PostgreSQL database

Key steps:

- 1) The user navigates through the app to a film's details page (from search results)
- 2) The backend fetches full film details from TMDB
- 3) The backend checks if the film is already in the user's viewing history or watchlist
- 4) The frontend displays the film details and adapts the action buttons accordingly
- 5) The user clicks "Log this film" and selects tags
- 6) The backend creates a viewing history entry and links the selected tags
- 7) A 201 Created response confirms the film has been logged



9.5. Film Recommendation

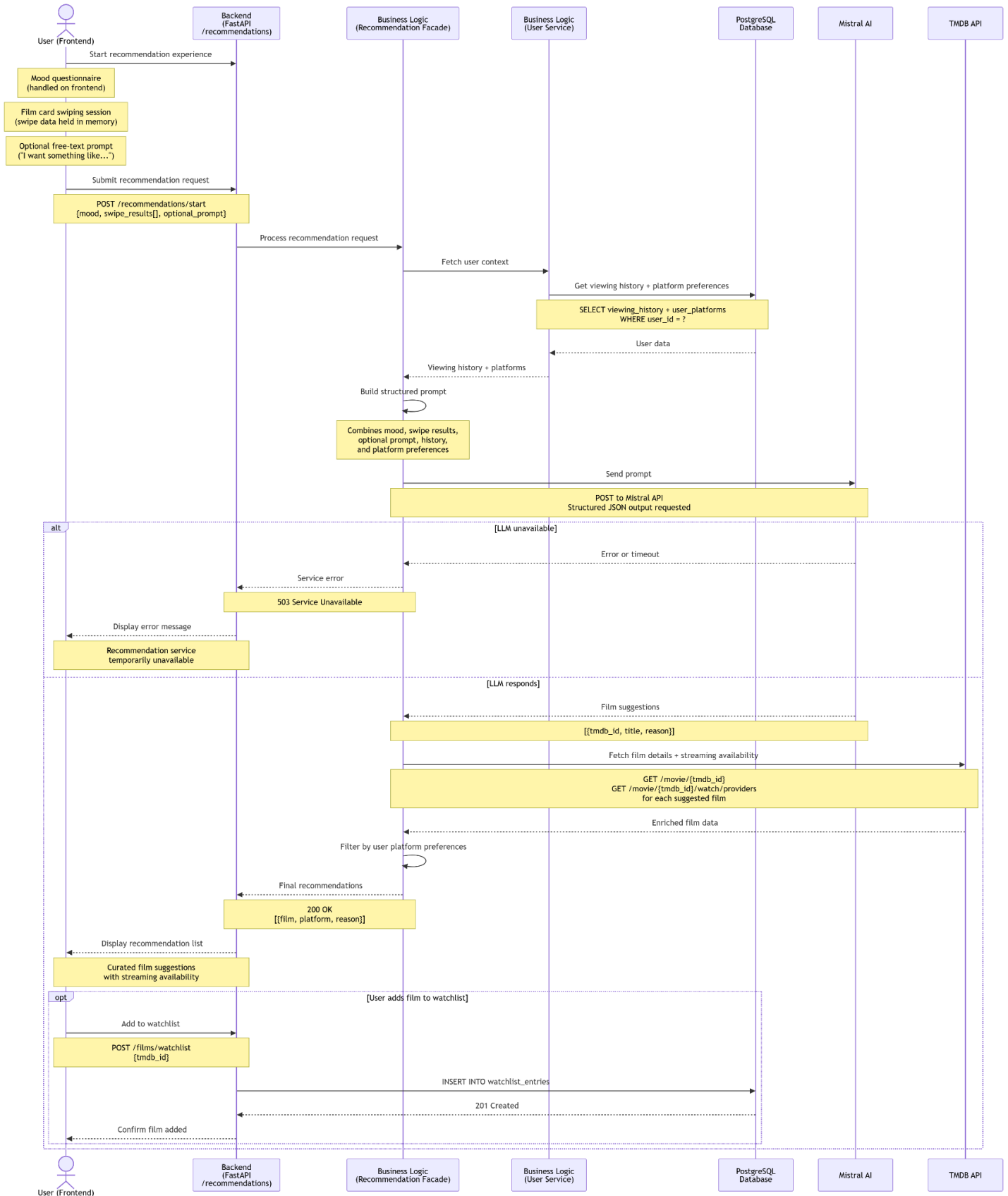
This sequence covers the full recommendation flow, from the mood questionnaire to the final film suggestions.

Actors:

- **User** - user interacting through the React frontend
- **Backend** - the `/recommendation/*` routes of the FastAPI
- **Business Logic - Recommendation Facade** - the Recommendation Facade of the FastAPI
- **Business Logic - User Service** - the User Service of the FastAPI
- **API** - the TMDB API
- **LLM** - the Mistral AI API
- **Database** - the PostgreSQL database

Key steps:

- 1) The user navigates from the home screen to the Recommendation experience
- 2) The user starts the recommendation experience
- 3) The user completes a mood questionnaire (handled entirely on the frontend)
- 4) The user swipes through film cards (right = interested, left = not interested)
- 5) The user optionally adds a free-text prompt before final submission
- 6) The backend retrieves the user's viewing history and platform preferences
- 7) The backend sends a structured prompt to the LLM
- 8) The LLM returns film suggestions
- 9) The backend enriches the suggestions with TMDB data and streaming availability
- 10) The frontend displays the final recommendation list



10. SCM and QA Strategy

This section defines the source control management and quality assurance strategies that will be applied throughout the development phase of CinéMood.

10.1. Source Control Management (SCM)

Version Control System

CinéMood uses Git for version control with GitHub as the remote repository host.

Repository: <https://github.com/FelixBesancon/holbertonschool-CineMood>

Branching Strategy

The project follows a simplified Gitflow branching model adapted for solo development:

Branch	Purpose	Rules
main	Production-ready code	Never pushed to directly. Only receives merges from develop after review.
develop	Integration branch	All feature branches merge here. Always in a working state.
feature/<name>	Individual features	Created from develop. Merged back into develop via pull request. Examples: feature/user-auth , feature/film-search , feature/recommendation-flow .
fix/<name>	Bug fixes	Created from develop (or main for critical hotfixes). Same merge process as feature branches.

Commit Conventions

Commits follow the Conventional Commits specification for clear, readable history:

```
<type>(<scope>): <short description>
```

Examples:

```
feat(auth): add JWT token generation on registration
fix(films): correct TMDB search query encoding
docs(readme): update setup instructions
test(users): add unit tests for validate_email
refactor(recommendations): extract prompt building to helper
function
```

Types used: **feat**, **fix**, **docs**, **test**, **refactor**, **chore**, **style**.

Pull Request Process

All merges into **develop** go through a pull request, even in solo development.

This enforces:

- A self-review checklist before merging (tests passing, no debug code, documentation updated).
- A clear description of what the branch does and what was tested.
- A link to the corresponding Trello/GitHub Projects task.

10.2. Quality Assurance (QA) Strategy

Testing Philosophy

The testing strategy for CinéMood focuses on the backend, where business logic is concentrated. The frontend is validated through manual testing and the Figma prototype. The goal is to reach a minimum of 50% test coverage on the backend codebase.

Backend Testing - pytest

The backend uses pytest as the primary testing framework.

Test type	What is tested	Tools
Unit tests	Individual functions and methods in isolation (validate_email, hash_password, verify_password, build_prompt, etc.)	pytest, unittest.mock
Integration tests	API endpoints with a test database - register, login, film search, log film, get history	pytest, httpx (FastAPI test client)
External API mocking	TMDB and Mistral AI responses are mocked to avoid real API calls in tests	pytest-mock, unittest.mock

Test files are located in the `/backend/tests/` directory and mirror the structure of the source code:

- **tests/test_auth.py** - registration and login flows
- **tests/test_films.py** - search, log, watchlist, history
- **tests/test_users.py** - profile and platform preferences
- **tests/test_recommendations.py** - recommendation flow and prompt building

Coverage is measured using **pytest-cov**:

```
pytest --cov=app --cov-report=html
```

The target is **50% minimum coverage**, focusing on the **service layer** and **critical endpoints**.

Frontend Testing - Manual + Postman

The frontend is tested manually against the running backend during development. Critical user flows are tested end-to-end:

- User registration and login
- Film search and detail view
- Film logging with tags
- Watchlist add and remove
- Full recommendation experience

Postman is used to test all internal API endpoints independently of the frontend. A Postman collection documents every endpoint with example requests and expected responses.

API Contract Testing

FastAPI's auto-generated Swagger UI (/docs) serves as an interactive API contract during development. Every new endpoint is verified in Swagger before being integrated with the frontend.

Error and Edge Case Coverage

The following edge cases are explicitly tested:

- Registration with a duplicate email (expected: **409 Conflict**)
- Login with wrong password (expected: **401 Unauthorized**, same message as user not found)
- Logging a film already in history (expected: handled by status check before logging)
- TMDB API unavailable (expected: **503 Service Unavailable** returned to frontend)
- LLM API unavailable (expected: **503 Service Unavailable** with user-facing message)

10.3. Development Pipeline

Stage	Description
Local development	Backend runs with uvicorn, frontend with Vite dev server. Both run locally with a local PostgreSQL instance.
Feature branch	Developer runs pytest before opening a pull request. Coverage report is checked manually.
Merge to develop	Pull request self-reviewed. Tests must pass. No debug code.
Merge to main	Final review before demo or submission. Main branch always reflects a working, presentable state.
Deployment	Deployment target and pipeline (e.g., Railway, Render, or VPS) to be defined in Stage 4.

11. Tech Stack Summary and Justifications

The following table summarises all technologies, languages, frameworks, tools, and external services used in the CinéMood project:

11.1. Technical Stack, Tools and APIs

Category	Technology	Role
Frontend language	JavaScript (JSX)	UI logic and component rendering
Frontend framework	React	Component-based UI architecture
CSS framework	Tailwind CSS	Utility-first styling and responsive layout
Frontend routing	React Router	Client-side navigation between pages
Backend language	Python	Server-side logic and API orchestration
Backend framework	FastAPI	REST API, data validation, async support
Data validation	Pydantic	Request/response schema validation (via FastAPI)
Authentication	JWT (JSON Web Tokens)	Stateless user authentication
Database	PostgreSQL	Relational data persistence
Frontend dev server	Vite	Local development server with hot-reload
Backend dev server	Uvicorn	ASGI server to run FastAPI locally
Backend testing	pytest + pytest-cov	Unit and integration tests, coverage measurement
API mocking	pytest-mock	Mocking TMDB and Mistral AI in tests
API testing	Postman	Manual endpoint testing and documentation
HTTP test client	httpx	FastAPI integration test client
Version control	Git + GitHub	Source control and project history
Task management	GitHub Projects + Trello	Sprint planning and task tracking

Category	Technology	Role
API documentation	Swagger UI (via FastAPI)	Auto-generated interactive API docs
Film metadata API	TMDB	Film data, posters, genres, cast, French support
Streaming availability API	TMDB Watch Providers	Platform availability by country
LLM API	Mistral AI	Mood-based film recommendation engine
Backup LLM API	Google Gemini	Fallback option during development
Design / Mockups	Figma	UI wireframes and prototype

11.2. Technical justifications:

Technical choice	Justification
Python for the backend	Python was selected as the backend language because it is the dominant language in the AI and machine learning ecosystem - libraries and SDKs for LLM integration (Mistral AI, LangChain, OpenAI) are all Python-native. This makes Python a natural fit for CinéMood's architecture, which relies on prompt engineering and external AI services, and keeps the door open for deeper ML integration in future versions. Python's readability and conciseness also accelerate development in a solo project context. It is additionally the language covered throughout the Holberton curriculum, which reduces context-switching and allows more focus on architecture and product quality.
FastAPI instead of Flask	FastAPI was chosen for its automatic request validation through Pydantic, built-in Swagger documentation, and native async support. These features are especially useful for CinéMood, where the backend must validate user input and orchestrate calls to TMDB and the LLM API.
Pydantic models	Pydantic enforces structured data validation on every incoming request and outgoing response. It automatically checks field types, required fields, and constraints at the API boundary, returning clear error messages if data is malformed. This eliminates manual validation code, reduces the risk of unexpected input reaching the business logic, and makes the API contract between the frontend and backend explicit and enforceable.

Technical choice	Justification
React for the frontend	React was selected because CinéMood's interface relies heavily on reusable and interactive components such as film cards, tag selectors, swipe cards, dashboard sections, and recommendation cards. Its component-based architecture improves maintainability and scalability.
Tailwind CSS	Tailwind CSS supports fast UI development and responsive design while allowing a custom visual identity. Unlike Bootstrap, it does not impose a generic visual style, which is important for CinéMood's cinematic atmosphere.
React Router or equivalent routing solution	The frontend is designed around several views such as dashboard, catalog, profile, and recommendation flow. A routing solution allows users to navigate between these views in a structured and fluid way.
Centralized API client	A centralized API client prevents duplicated fetch logic across components. It also keeps authentication headers, error handling, and response parsing consistent throughout the frontend.
JWT authentication	JWT enables stateless authentication for the REST API. The backend does not need to store session data, and each authenticated request can include the token in the request headers.
PostgreSQL database	PostgreSQL was chosen because CinéMood's data is relational: users have watchlists, viewing history entries, tags, and platform preferences. A relational database provides strong consistency, foreign keys, joins, and cascade operations.
Relational model instead of NoSQL	A document-oriented database would make highly connected data harder to query and maintain. PostgreSQL better fits the relationships between users, history entries, tags, platforms, and watchlist entries.
No local films table	CinéMood does not store full film metadata locally. TMDB remains the source of truth, and only the <code>tmdb_id</code> is stored in the database. This avoids duplicated or outdated film data.
Film DTO	The Film object is designed as a temporary Data Transfer Object populated from TMDB responses. It carries film data between the backend and frontend without being persisted in the local database.
Repository pattern	The repository pattern isolates database queries from business logic. This makes the backend easier to maintain, test, and evolve if the persistence layer changes.

Technical choice	Justification
Facade pattern for recommendations	The recommendation flow requires user data, viewing history, mood input, swipe results, TMDB metadata, streaming availability, and LLM output. A facade hides this complexity behind one clean interface.
REST API design	REST was selected because it is simple, stateless, resource-oriented, and well suited for communication between a React frontend and a FastAPI backend using JSON.
TMDB API for film metadata	TMDB provides film titles, posters, synopses, genres, cast, crew, and runtime with a generous free tier. It also supports multilingual data, which keeps the door open for a future French version.
TMDB Watch Providers for streaming availability	TMDB Watch Providers is included in the same API ecosystem, reducing cost and integration complexity. It provides country-based streaming availability, including France.
Mistral AI as primary LLM provider	Mistral AI was selected because it is a European provider, offers competitive pricing, and aligns with GDPR-conscious architecture. It is suitable for prompt-based film recommendation without training a custom model.
Google Gemini as backup LLM provider	Gemini is documented as a backup option because of its generous free tier during development. It can help preserve Mistral credits while testing the recommendation flow.
Prompt engineering instead of model training	The MVP does not require training a custom model. Prompt engineering is simpler, faster, cheaper, and more realistic for a student MVP while still allowing personalized recommendations.
Predefined emotional/contentual tags	Fixed tags make user feedback easier to structure, compare, and reuse in recommendations. They also avoid the complexity of managing unlimited custom user-generated tags in the MVP.
PrestigeTier enumeration	The prestige tier provides a simple global rating system that complements emotional tags. Using an enumeration keeps possible values controlled and consistent.
Figma for mockups	Figma was used to prototype the main screens and validate the user journey before implementation. It helps connect user stories to concrete interface decisions.
Git and GitHub	Git and GitHub provide version control, project history, and a safe workflow for developing the MVP incrementally.

Technical choice	Justification
Swagger UI via FastAPI	Swagger UI is automatically generated by FastAPI and allows endpoints to be tested directly in the browser during development. This improves debugging and API documentation quality.
